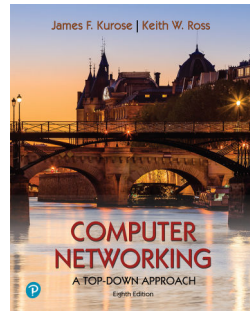


第3章 传输层

中国科学技术大学
自动化系 郑烜
改编自Jim kurose, Keith Ross

2025中科大网络



*Computer Networking: A
Top-Down Approach*
8th edition
Jim Kurose, Keith Ross
Pearson, 2020

Transport Layer: 3-1

2025中科大网络

传输层：概述

目标:

- 理解传输层服务背后的原理
 - 多路复用、解复用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 学习互联网传输层协议实例
 - UDP：无连接传输
 - TCP：面向连接的可靠传输
 - TCP拥塞控制

Transport Layer: 3-2

传输层：提纲

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



2025中科大网络

Transport Layer: 3-3

传输层：提纲

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



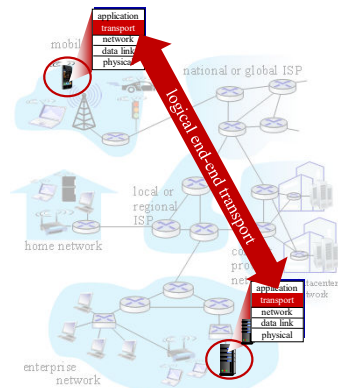
2025中科大网络

Transport Layer: 3-4

传输层服务和协议

2025中科大网络

- 为不同主机上运行着的应用进程提供它们之间的**逻辑通信**
- 传输层协议在端系统中运行
 - 发送端：将应用层报文分割成**报文段**，交给网络层
 - 接收端：重组这些报文段形成报文，交给应用层
- 互联网应用可用的2个传输层协议
 - TCP, UDP



Transport Layer: 3-5

传输层 vs 网络层服务和协议

2025中科大网络



— 家庭间通信的类比: —

Ann 大家庭中的12个小孩发送信件给Bill大家庭中的12个小孩

- 主机=家庭居住的房子
- 进程=小孩
- 应用报文=信封中的信件
- 传输层协议=在屋内扮演者复用解复用角色的Ann和Bill
- 网络层协议=邮政服务

Transport Layer: 3-6

传输层 vs. 网络层 服务和协议

2025中科大网络

- 网络层**: 在**主机**之间提供逻辑的通信
- 传输层**: 在**进程**之间提供逻辑的通信
 - 依赖于网络层提供的服务，加强了网络层提供的服务

- 有些服务是可以加强的：不可靠 -> 可靠；安全
- 但有些服务是不可以被加强的：带宽，延迟

— 家庭间通信的类比: —

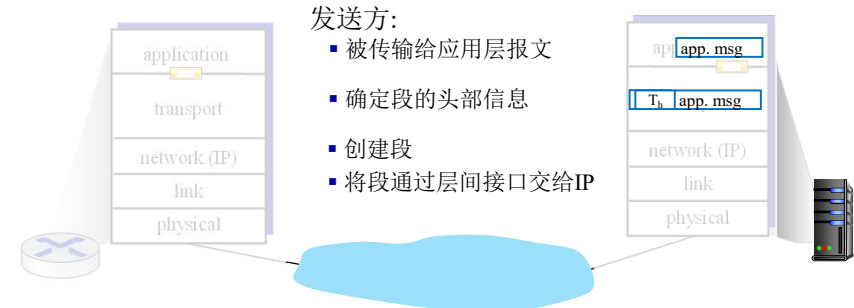
Ann 大家庭中的12个小孩发送信件给Bill大家庭中的12个小孩

- 主机=家庭居住的房子
- 进程=小孩
- 应用报文=信封中的信件
- 传输层协议=在屋内扮演者复用解复用的Ann和Bill
- 网络层协议=邮政服务

Transport Layer: 3-7

传输层运行

2025中科大网络



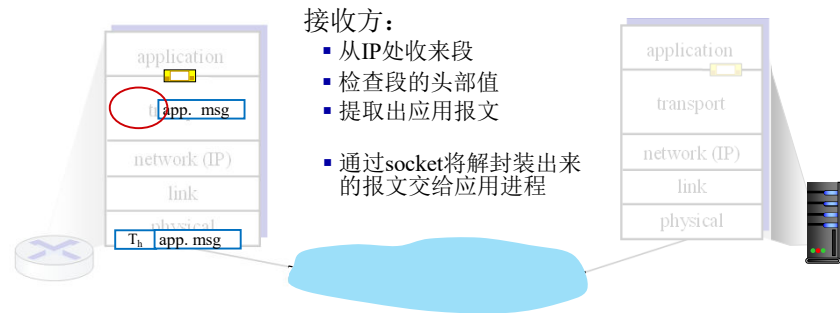
发送方:

- 被传输给应用层报文
- 确定段的头部信息
- 创建段
- 将段通过层间接口交给IP

Transport Layer: 3-8

传输层运行

2025中科大网络

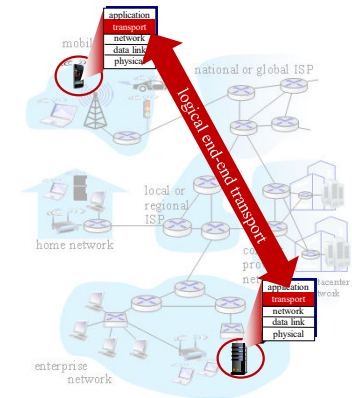


Transport Layer: 3-9

2个主要的互联网传输层协议

2025中科大网络

- **TCP: Transmission Control Protocol**
 - 可靠、保序的（字节流）传输
 - 拥塞控制
 - 流量控制
 - 连接建立
- **UDP: User Datagram Protocol**
 - 不可靠、不保序的（数据报）传输
 - 没有在尽力而为的IP服务上提供额外的扩展
- 二者都不提供
 - 延迟的保证
 - 带宽的保证
 - 安全的保证



Transport Layer: 3-10

传输层：提纲

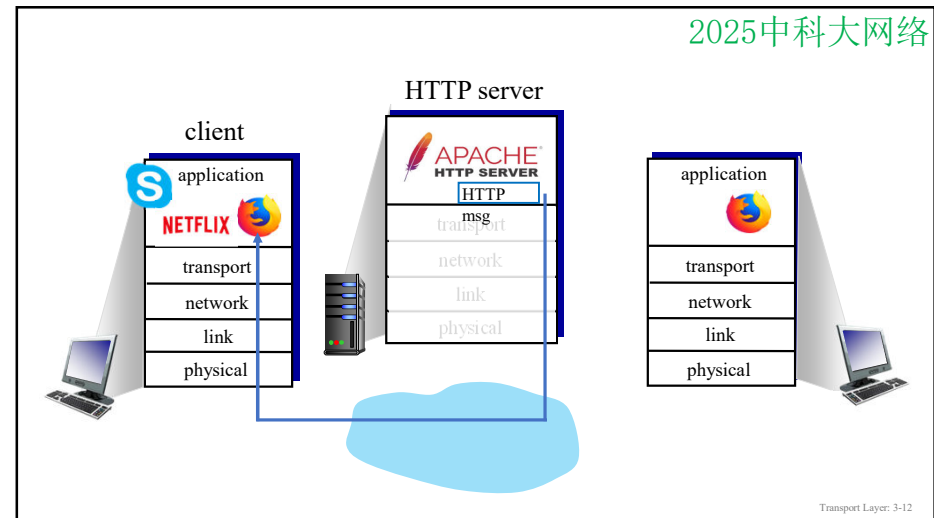
2025中科大网络

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



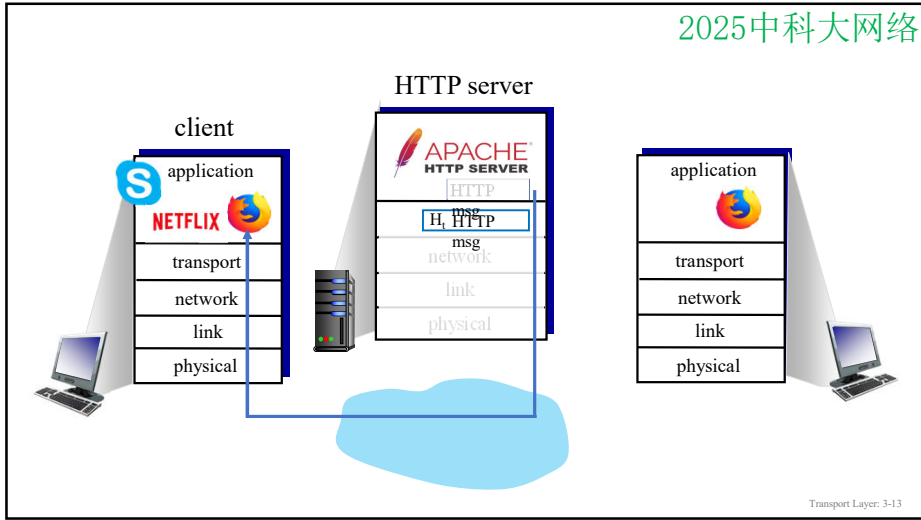
Transport Layer: 3-11

2025中科大网络

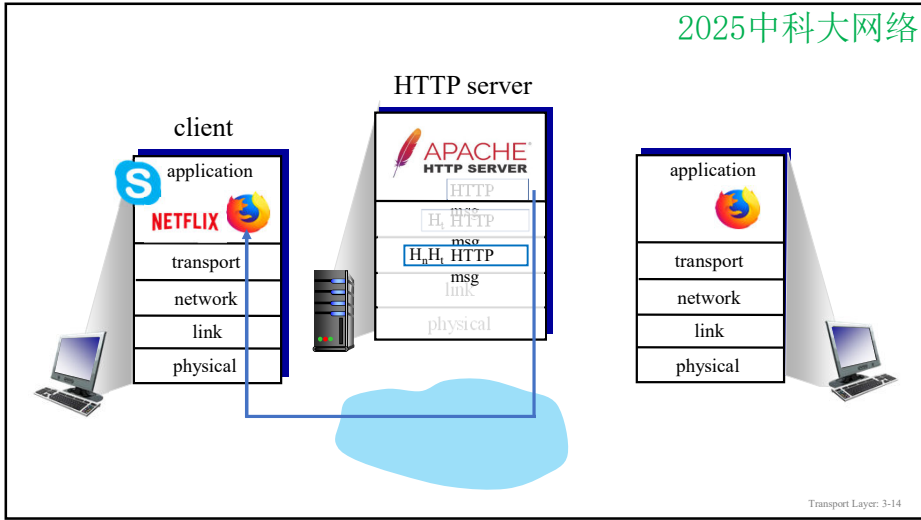


Transport Layer: 3-12

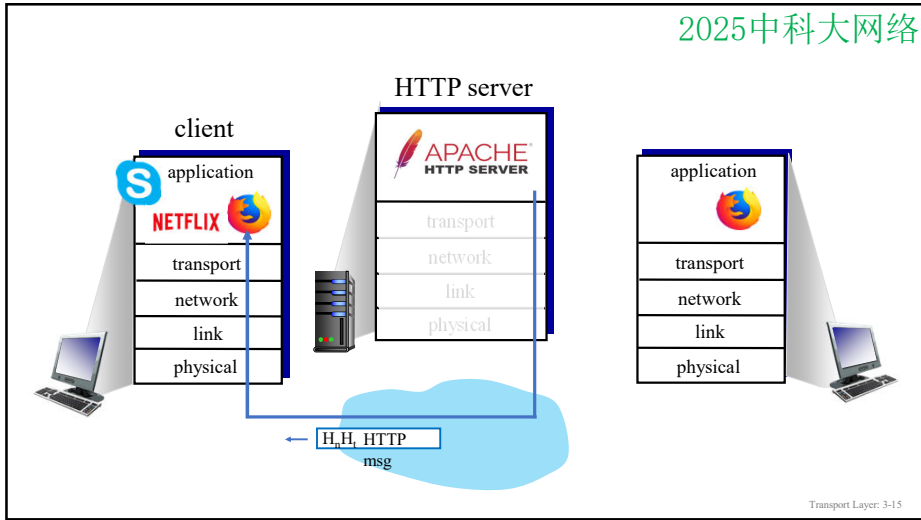
2025中科大网络



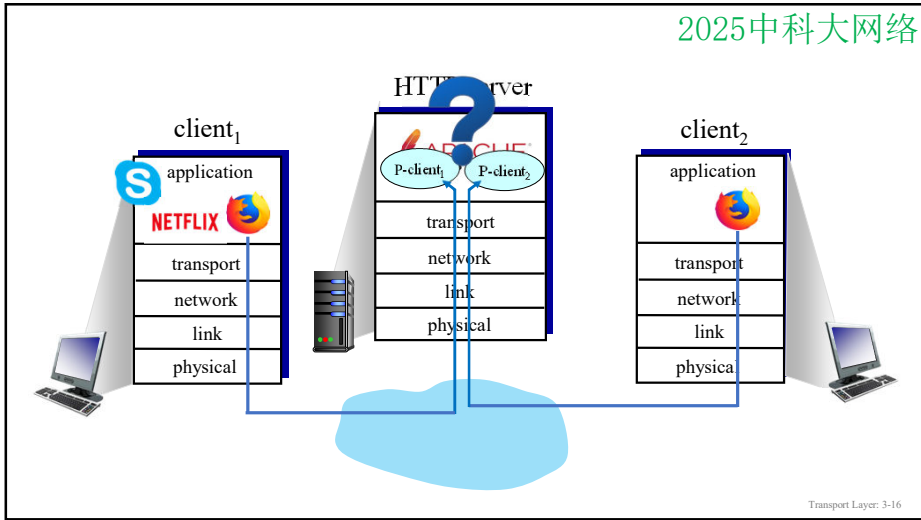
2025中科大网络



2025中科大网络



2025中科大网络

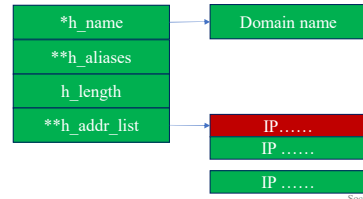


关键数据结构

```
struct sockaddr_in {
    short sin_family; //AF_INET
    u_short sin_port; // port
    struct in_addr sin_addr;
    // IP address, unsigned long
    char sin_zero[8]; // align
};
```

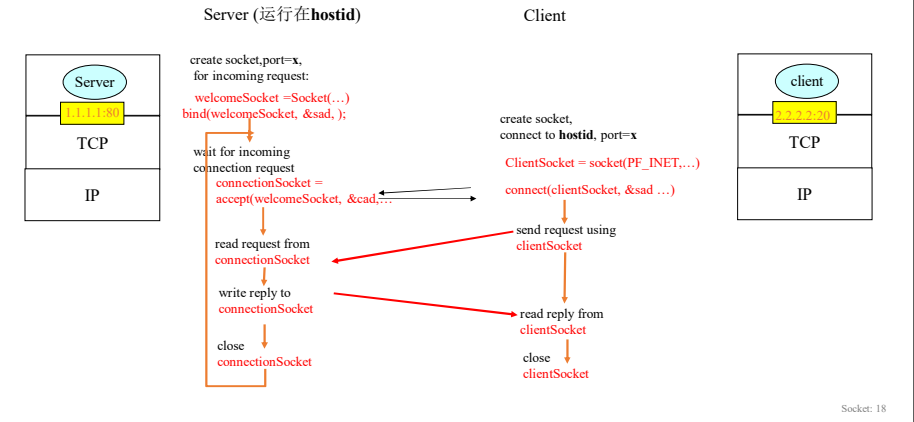


```
struct hostent
{
    char *h_name;
    char **h_aliases;
    int h_addrtype;
    int h_length; /*地址长度*/
    char **h_addr_list;
    #define h_addr h_addr_list[0];
};
```



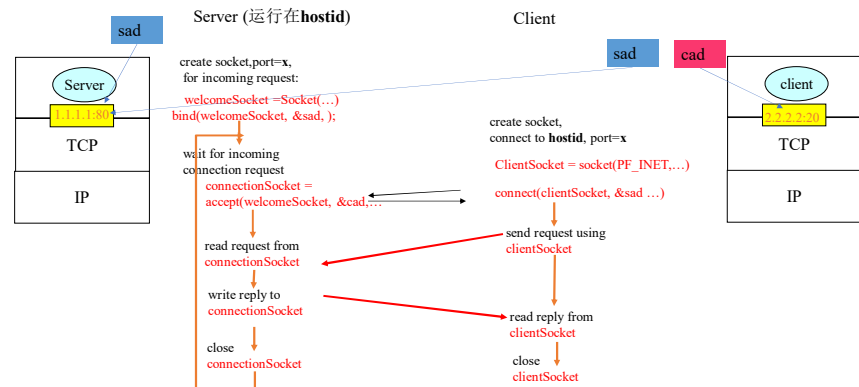
Socket: 17

TCP封装解封装和复用解复用过程



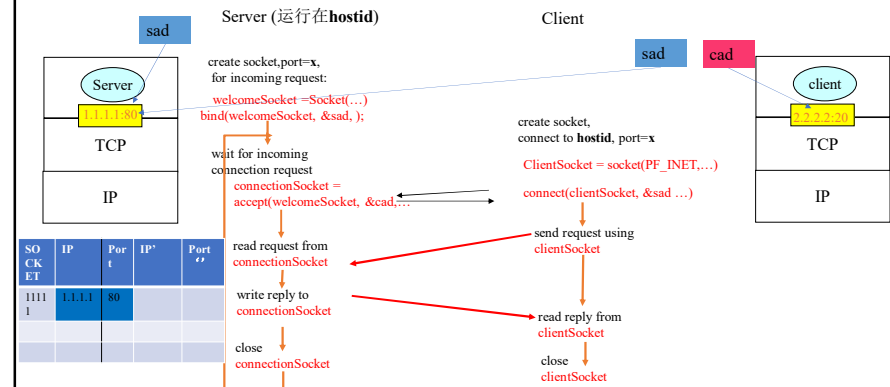
Socket: 18

TCP封装解封装和复用解复用过程



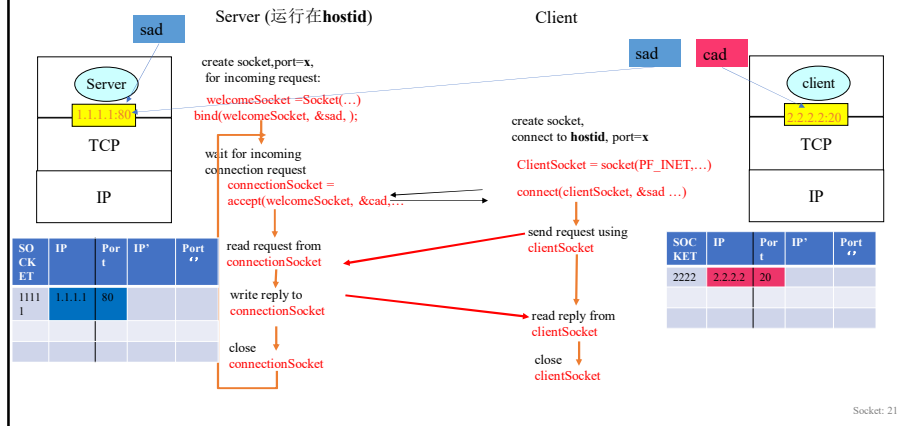
Socket: 19

TCP封装解封装和复用解复用过程

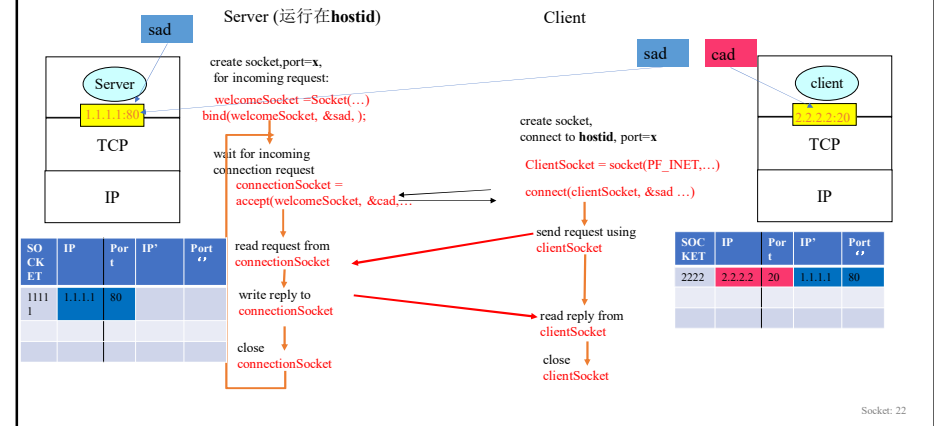


Socket: 20

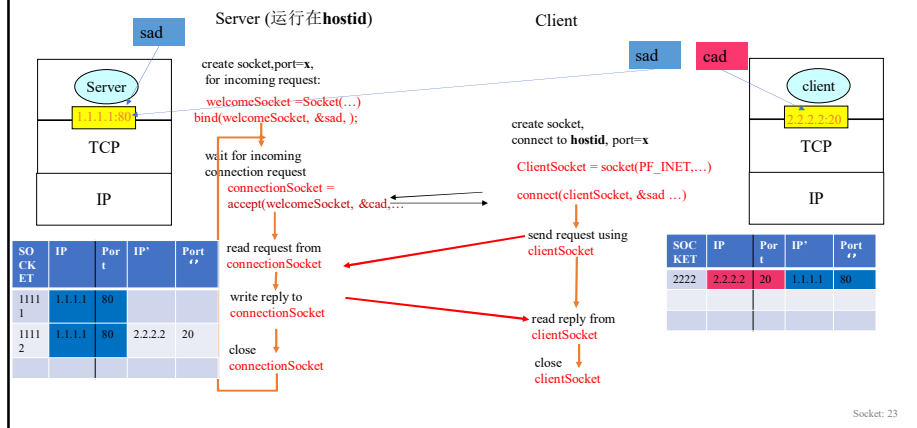
TCP封装解封装和复用解复用过程 2025中科大网络



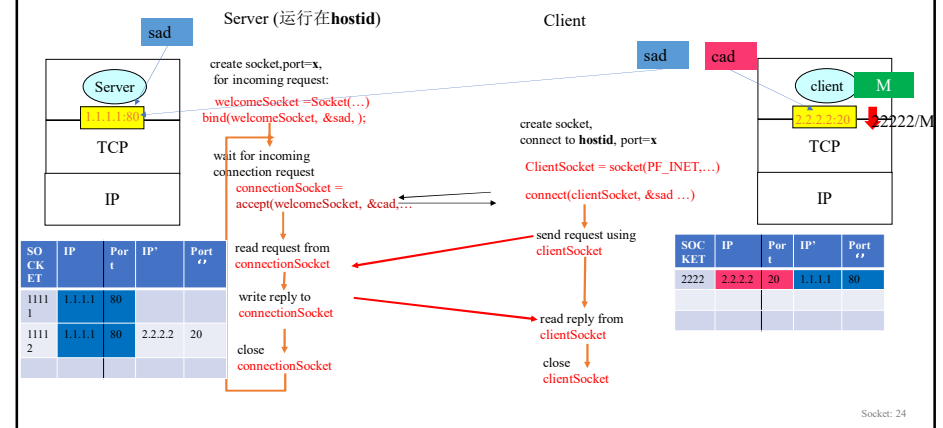
TCP封装解封装和复用解复用过程 2025中科大网络



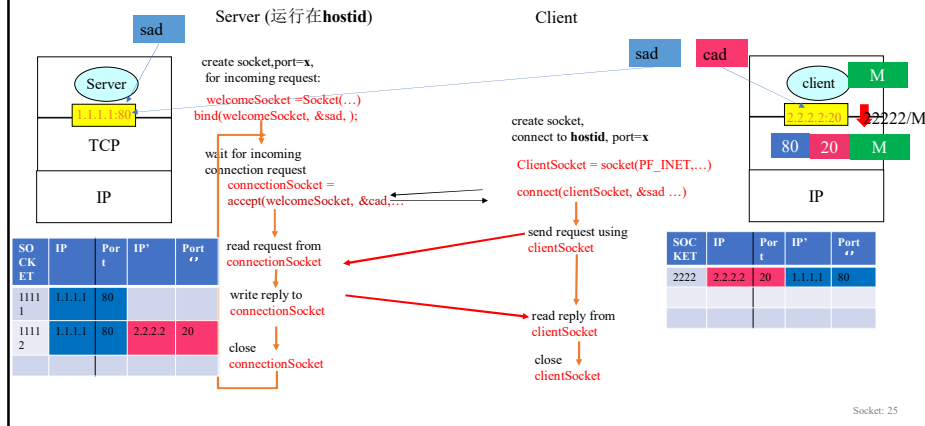
TCP封装解封装和复用解复用过程 2025中科大网络



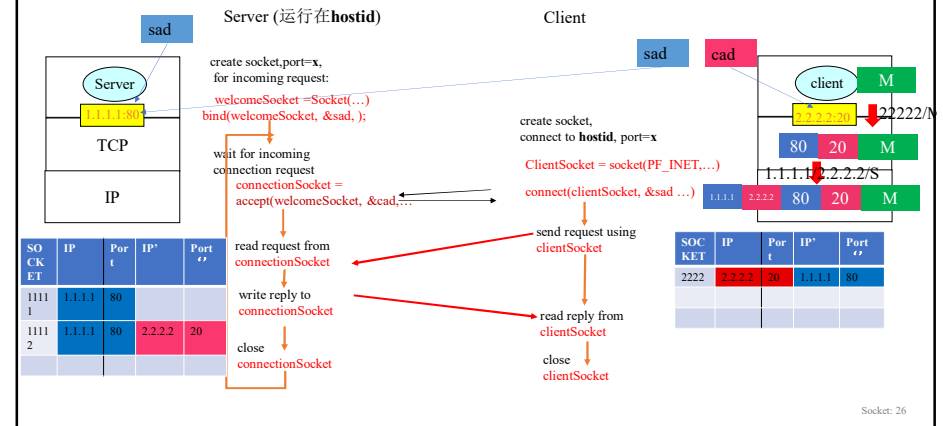
TCP封装解封装和复用解复用过程 2025中科大网络



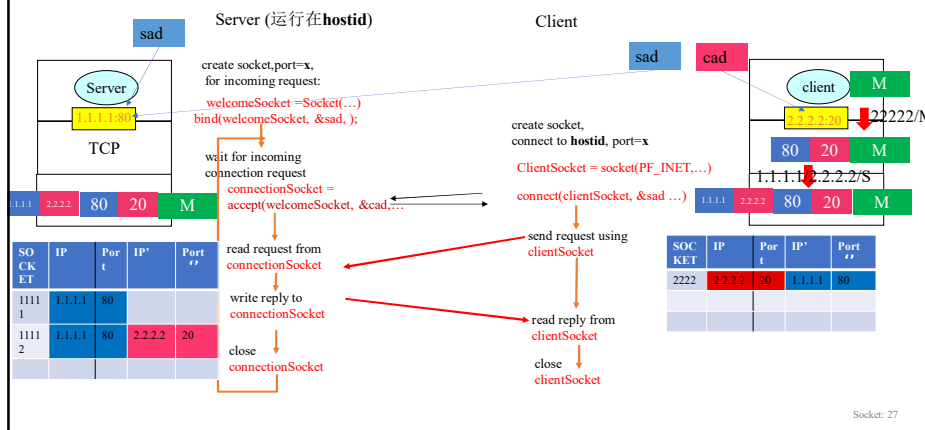
TCP封装解封装和复用解复用过程 2025中科大网络



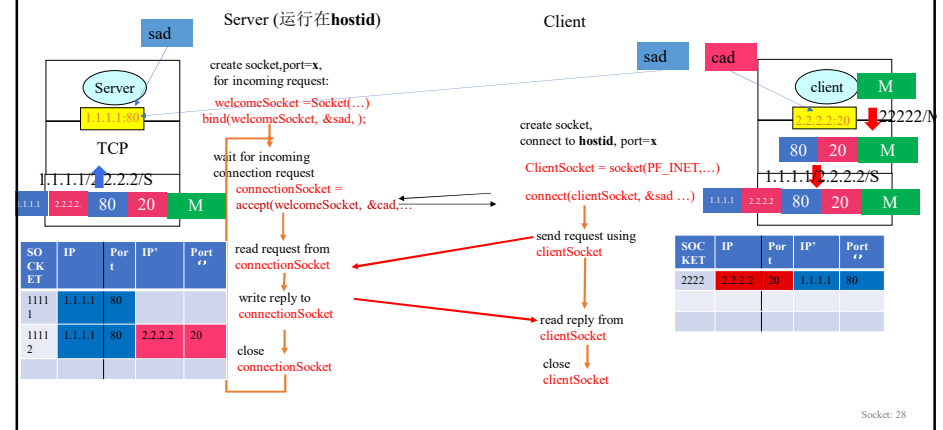
TCP封装解封装和复用解复用过程 2025中科大网络



TCP封装解封装和复用解复用过程 2025中科大网络

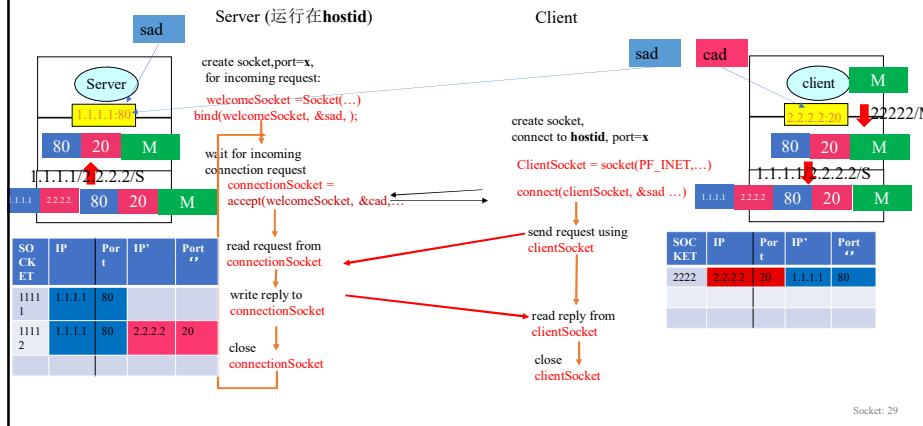


TCP封装解封装和复用解复用过程 2025中科大网络



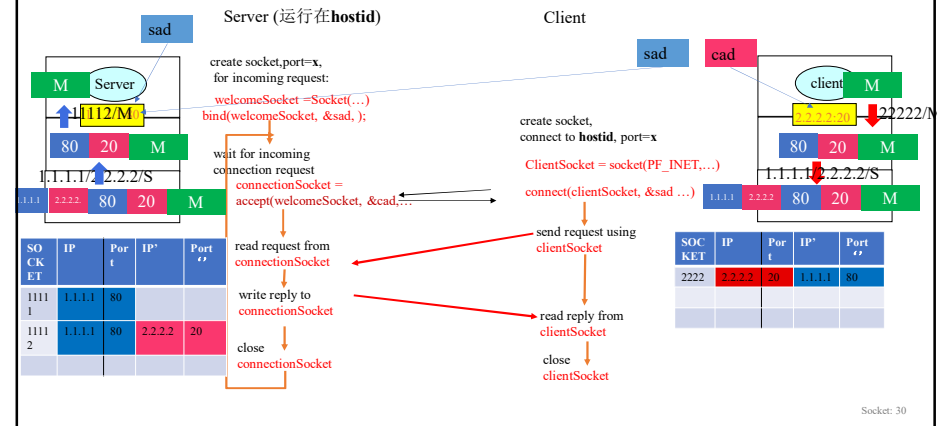
TCP封装解封装和复用解复用过程

2025中科大网络



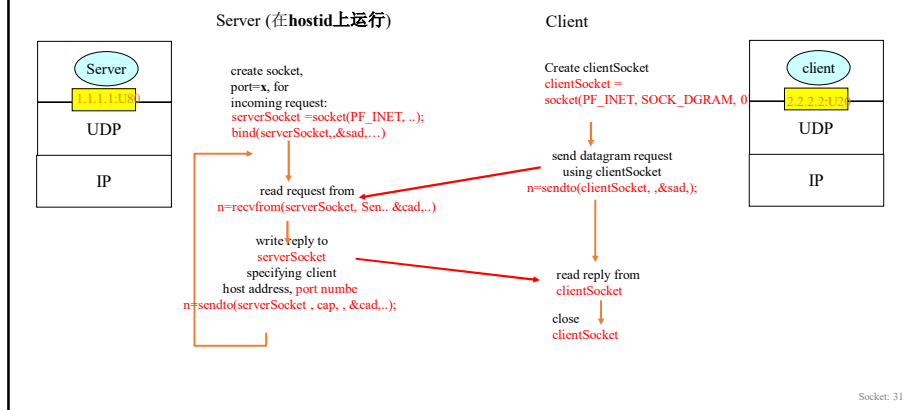
TCP封装解封装和复用解复用过程

2025中科大网络



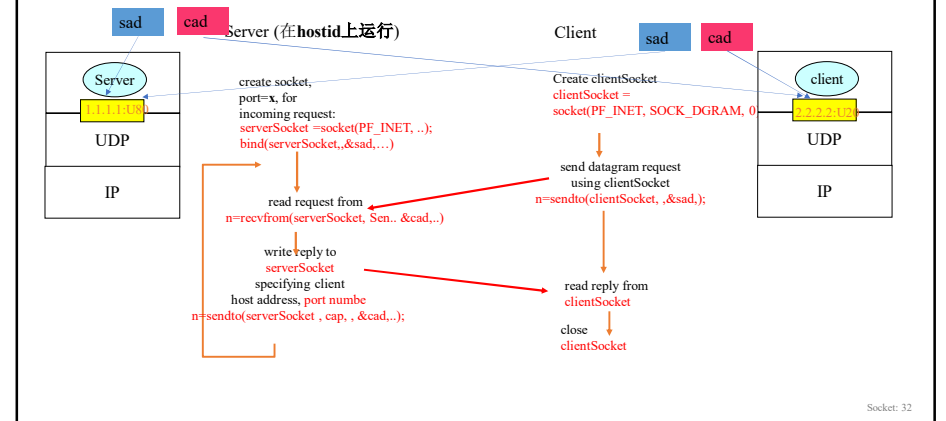
UDP封装解封装和复用解复用过程

2025中科大网络



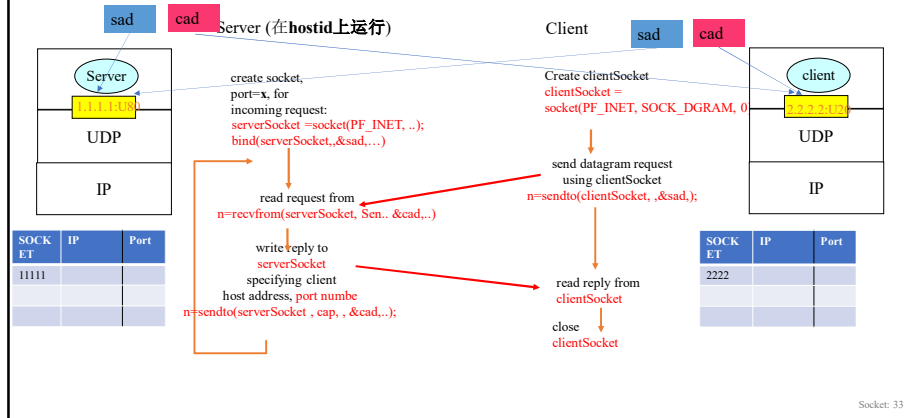
UDP封装解封装和复用解复用过程

2025中科大网络



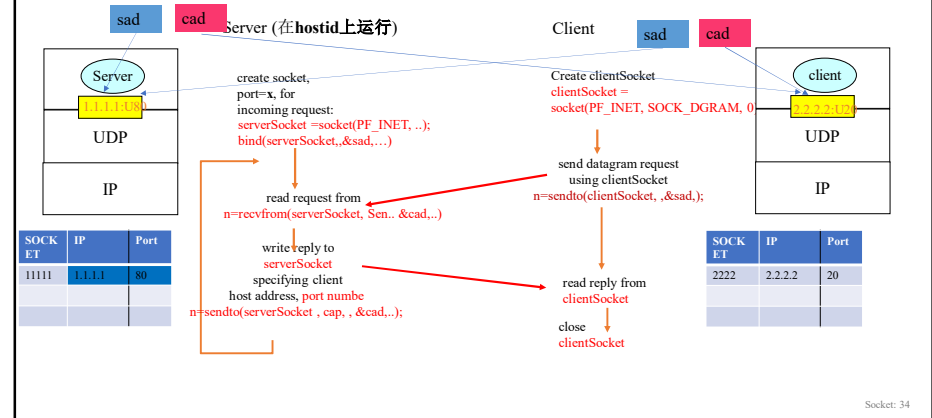
UDP封装解封装和复用解复用过程

2025中科大网络



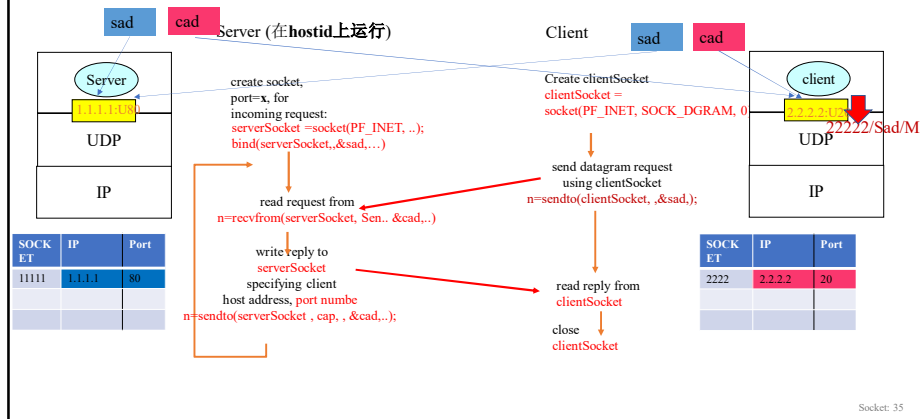
UDP封装解封装和复用解复用过程

2025中科大网络



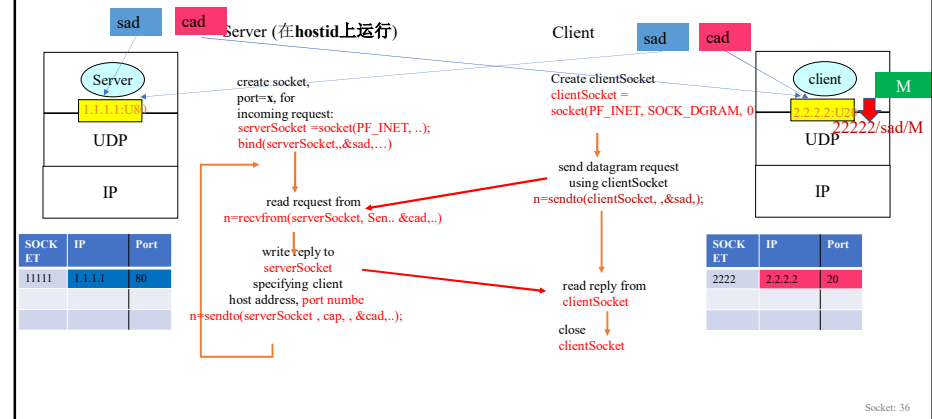
UDP封装解封装和复用解复用过程

2025中科大网络



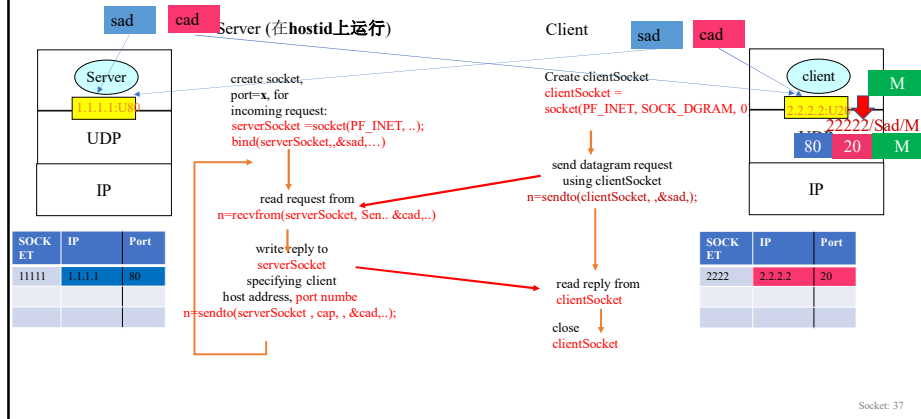
UDP封装解封装和复用解复用过程

2025中科大网络



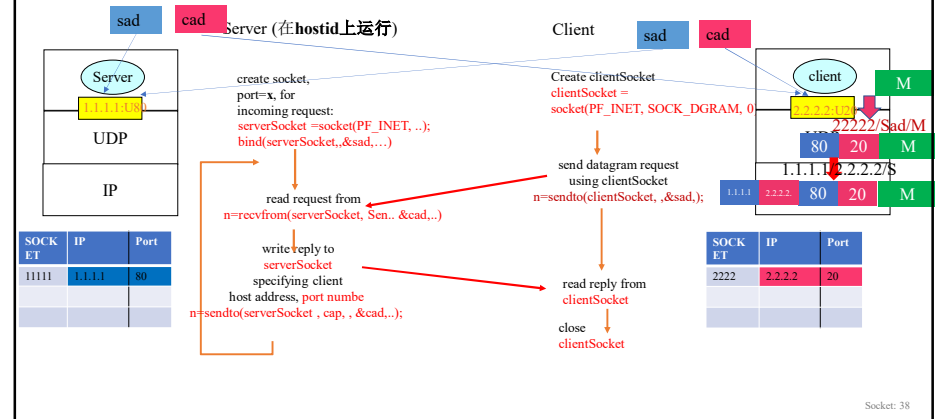
UDP封装解封装和复用解复用过程

2025中科大网络



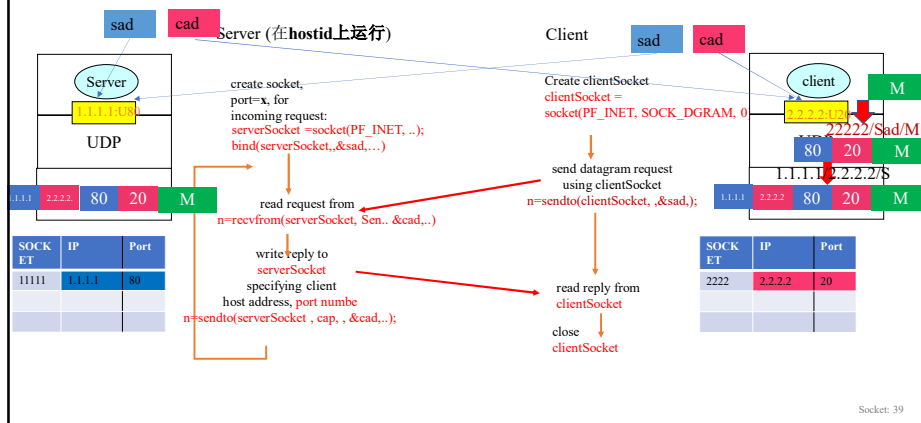
UDP封装解封装和复用解复用过程

2025中科大网络



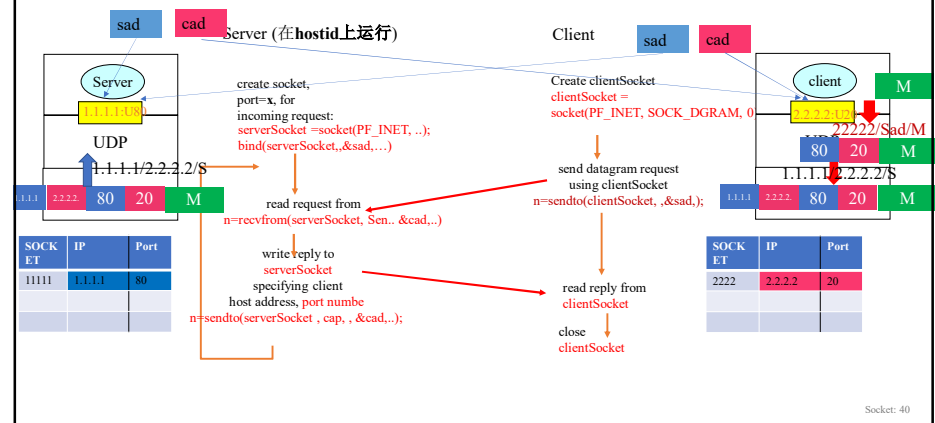
UDP封装解封装和复用解复用过程

2025中科大网络



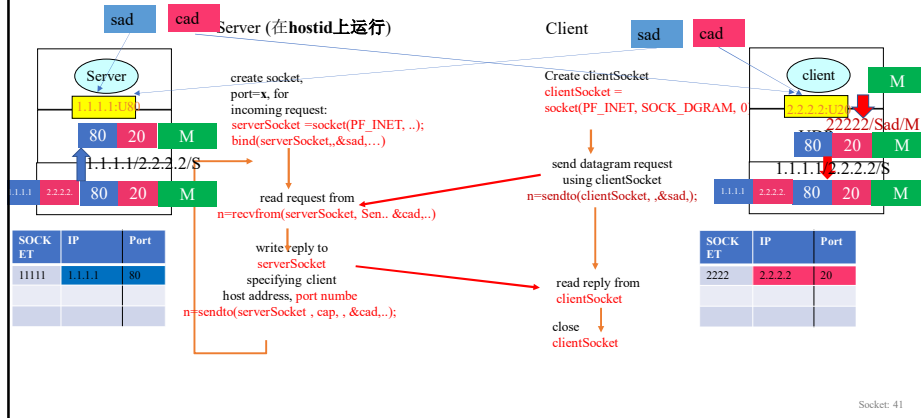
UDP封装解封装和复用解复用过程

2025中科大网络



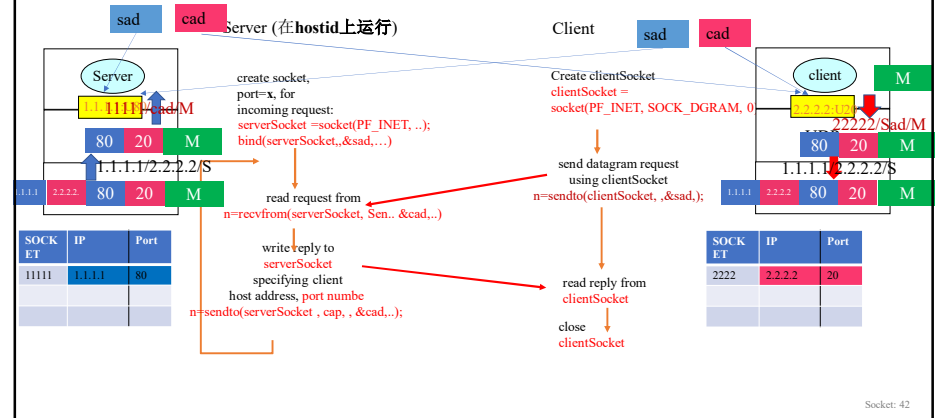
UDP封装解封装和复用解复用过程

2025中科大网络



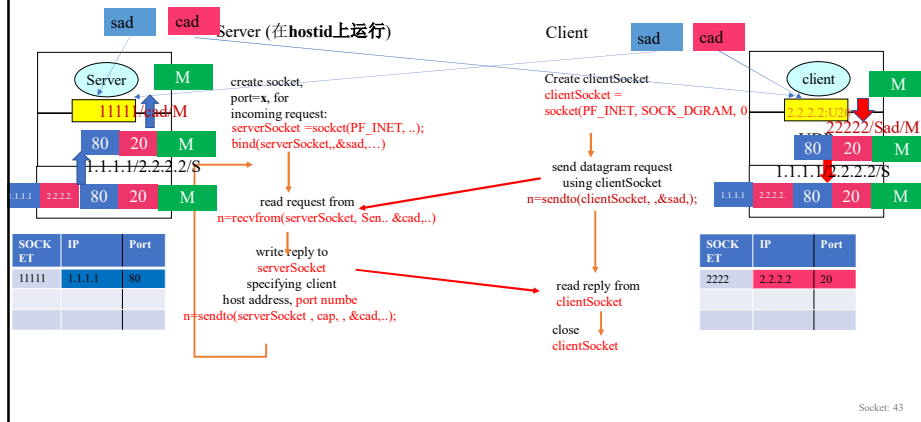
UDP封装解封装和复用解复用过程

2025中科大网络



UDP封装解封装和复用解复用过程

2025中科大网络



多路复用/解复用

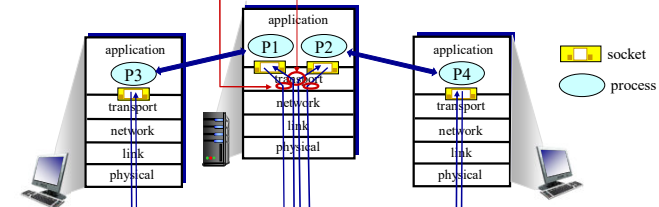
2025中科大网络

发送方多路复用:

TCP实体处理来自多个socket的数据, 增加传输层头部 (之后用于解复用)

接收方解复用

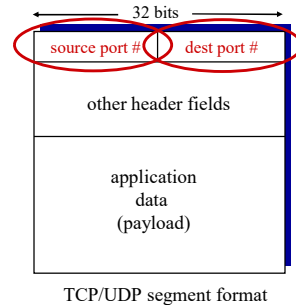
TCP实体使用头部信息, 将收到的报文段交付给正确的socket



解复用工作原理

2025中科大网络

- 主机收到IP数据报
 - 每个数据报包含有源IP和目标IP地址信息
 - 每个IP数据报携带传输层的段
 - 每个段携带源端口号和目标端口号
- 主机使用IP地址和端口号，来确定将段中的数据部分交给合适的socket
 - TCP综合四元组
 - UDP综合目标IP、目标端口号二元组



Transport Layer: 3-45

无连接的解复用

2025中科大网络

回忆:

- 创建socket时，必须指定本地的端口号:

```
DatagramSocket mySocket1 = new
  DatagramSocket(12534);
```

- 当通过Socket向目标传输的UDP数据报时，必须指定

- 对方（目标）IP地址
- 对方（目标）端口号

当接收端主机接收到UDP数据包时:

- 检查段中的目标端口号
- 将UDP数据报导向守候该端口号的socket

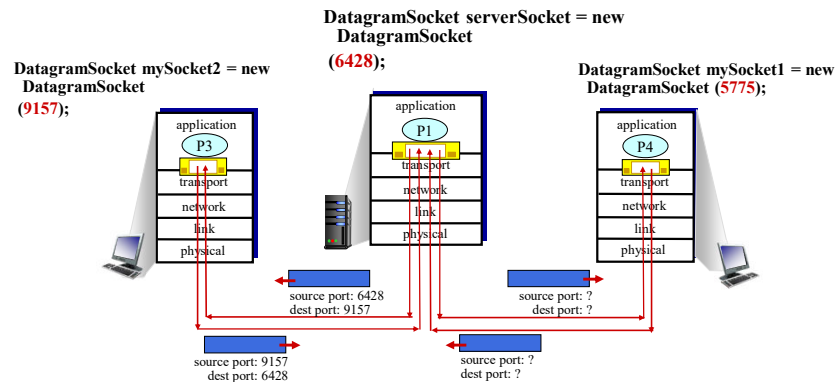


只要具有同样目标端口号的IP/UDP数据报，即使有不同的源IP、（和/或）源端口号，都被导向到接收主机的同一个socket

Transport Layer: 3-46

无连接的解复用：一个实例

2025中科大网络



Transport Layer: 3-47

面向连接的多路复用

2025中科大网络

- TCP socket被4元组标识:

- 源IP地址
- 源端口号
- 目标IP地址
- 目标端口号

- 服务器（在同样一个端口上）可以同时支持多个socket

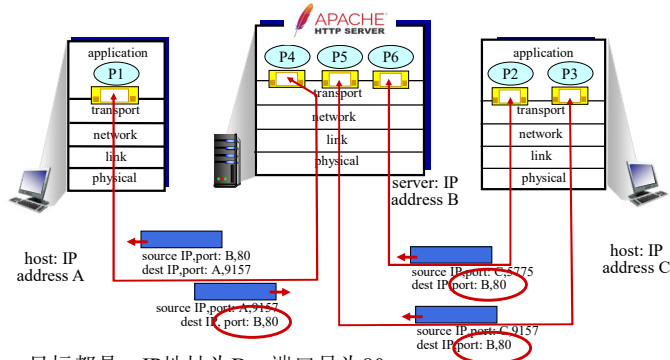
- 每个socket由自己的4元组标识
- 每个socket和不同的连接着的远程客户端（不同的IP地址和TCP端口）关联

- 解复用: 接收端使用全部的4个值来将段交给给正确的socket

Transport Layer: 3-48

面向连接的解复用：例子

2025中科大网络



3个段，目标都是：IP地址为B，端口号为80
但是被解复用到不同的socket上

Transport Layer: 3-49

总结

2025中科大网络

- 复用、解复用：基于(TCP or UDP)段，IP数据报的头部字段
- **UDP**：在接收端采用目标端口号进行解复用
- **TCP**：采用4元组进行解复用：源IP地址和目标IP地址，以及源和目标端口号
- 复用/解复用的技术在所有层次都会有

Transport Layer: 3-50

传输层：提纲

2025中科大网络

- 传输层服务
- 多路复用解复用
- **无连接的传输：UDP**
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



Transport Layer: 3-51

UDP: User Datagram Protocol

2025中科大网络

- “no frills,” “bare bones”互联网传输层协议
- “尽力而为”的服务，UDP数据报可能会：
 - 丢失
 - 交付给应用的报文产生乱序
- **无连接**：
 - UDP之上的发送方和接收方应用无需建立连接
 - 每个UDP数据报独立被处理，与其他UDP数据报相互独立

为什么需要 UDP?

- 没有连接建立 (连接建立需要耗费RTT的延迟)
- 简单：在发送方和接收方无需维护连接的状态
- 小的头部开销
- 没有拥塞控制
 - UDP不受拥塞控制制约，可以按照自己的速率往网络中发送
 - 在拥塞时还可以往外发送
 - 适合某些实时流媒体应用

Transport Layer: 3-52

UDP: User Datagram Protocol

2025中科大网络

- UDP 被用于:
 - 流媒体应用 (可以忍受丢失, 但是对于带宽敏感)
 - DNS
 - SNMP
 - HTTP/3
- 如果需要在UDP之上进行可靠数据传输(e.g., HTTP/3):
 - 在应用层增加必须的**可靠性恢复**机制
 - 在应用层增加**拥塞控制**机制

Transport Layer: 3-53

UDP: User Datagram Protocol [RFC 768]

2025中科大网络

RFC 768

INTERNET STANDARD

J. Postel

ISI

28 August 1980

User Datagram Protocol

Introduction

This User Datagram Protocol (UDP) is defined to make available a datagram mode of packet-switched computer communication in the environment of an interconnected set of computer networks. This protocol assumes that the Internet Protocol (IP) [1] is used as the underlying protocol.

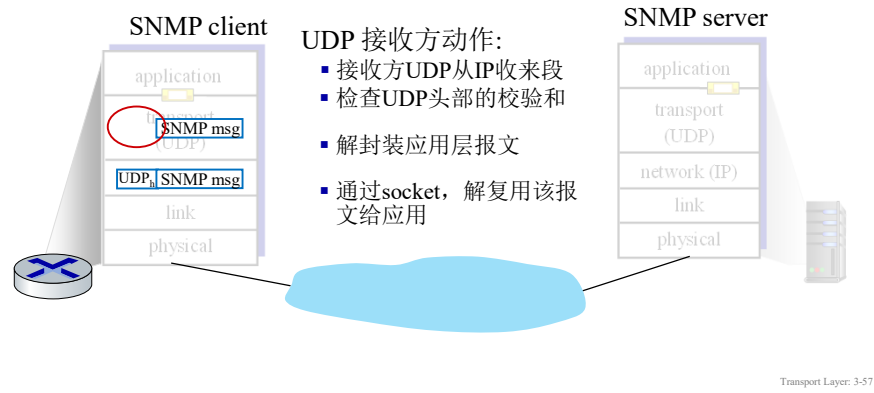
This protocol provides a procedure for application programs to send messages to other programs with a minimum of protocol mechanism. The protocol is transaction oriented, and delivery and duplicate protection are not guaranteed. Applications requiring ordered reliable delivery of streams of data should use the Transmission Control Protocol (TCP) [2].

Format

	0	7	8	15	16	23	24	31
Source Port	Destination Port							
Length				Checksum				
data octets ...								

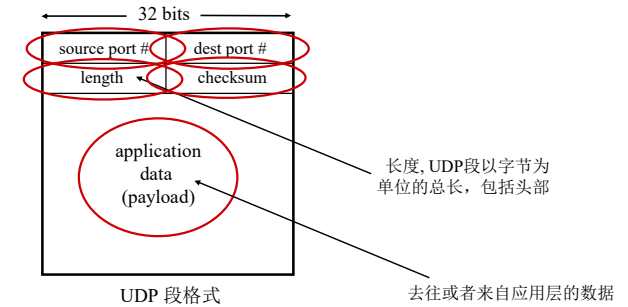
UDP: 传输层面的动作

2025中科大网络



UDP 段的头部

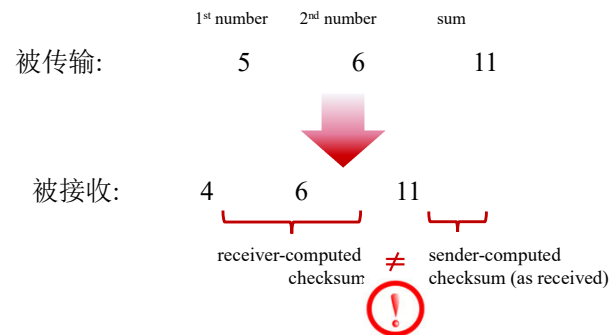
2025中科大网络



UDP 校验和

2025中科大网络

目标: 在被传输的UDP段中检查出可能的出错 (如: 位的反转)



UDP 校验和

2025中科大网络

目标: 在被传输的UDP段中检查出可能的出错 (例如: 位反转)

发送方:

- 将UDP传输内容 (包括UDP头部字段和IP地址) 当成是16比特的整数序列
- 校验和:** 这些段内容的和 (1的补码和)
- 将校验和的值填入UDP校验和字段

接收方:

- 计算接收到段的校验和
- 判断计算出来的校验和是否等于校验和字段的值:
 - 不相等-出错
 - 相等-大概率没出错, 但是:
 - 也可能没有错误被检查出来, 有漏检错误, 后面详述

Transport Layer: 3-60

互联网校验和：例子

2025中科大网络

例子：2个16比特整数

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
回卷	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1
和	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0
校验和	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1

注：进位回卷，在进行加法运算时，必须将最高位进位加到整数的最低位，再进行求和

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Transport Layer: 3-61

互联网校验和：保护能力有限

2025中科大网络

例子：2个16比特整数

	1 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0	0 1
	1 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1	1 0
和	1 1 0 1 1 1 0 1 1 1 0 1 1 1 0 1 1	
校验和	1 0 1 1 1 0 1 1 1 0 1 1 1 1 0 0	
校验和	0 1 0 0 0 1 0 0 0 1 0 0 0 0 1 1	

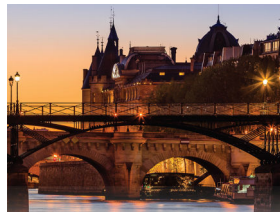
即使是发生了错误（位的反转），校验和也检查不出，与不出错完全相同！

Transport Layer: 3-62

传输层：提纲

2025中科大网络

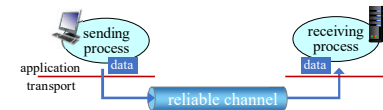
- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



Transport Layer: 3-63

可靠数据传输原理

2025中科大网络



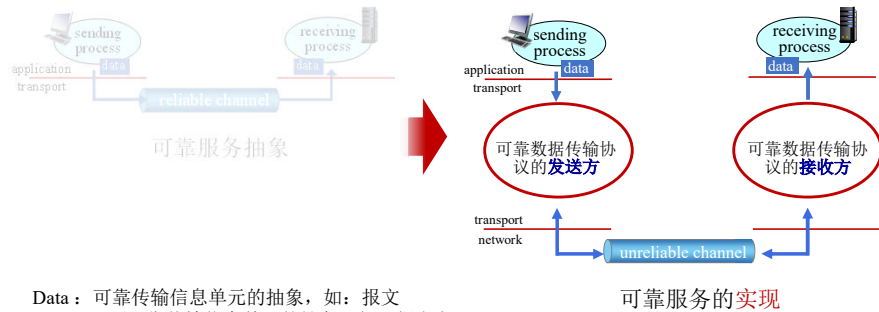
可靠服务的抽象

- 可靠数据传输可在很多层实现
- TCP向应用提供可靠的报文（字节流）传输服务
- 某些物理网络的链路层向上层提供可靠数据传输服务

Transport Layer: 3-64

可靠数据传输原理

2025中科大网络

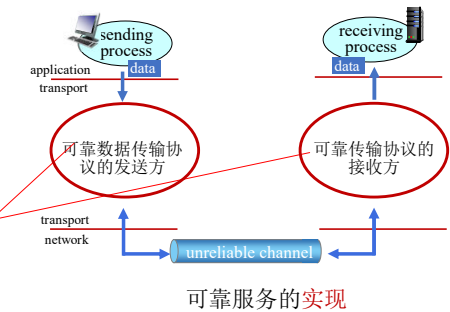


Transport Layer: 3-65

可靠数据传输原理

2025中科大网络

可靠数据传输协议的复杂性将
 (强烈地) 依赖于信道的不可
 靠程度 (丢失、损坏、数据乱
 序的情况?)



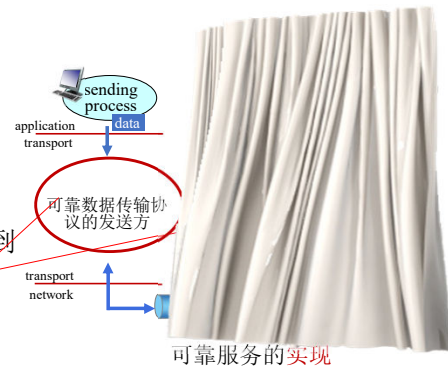
Transport Layer: 3-66

可靠数据传输原理

2025中科大网络

发送方和接收方都不知道对方的
 状态, 例如: packet是否被接收到了
 吗? (异地)

- 除非通过packet交换控制信息



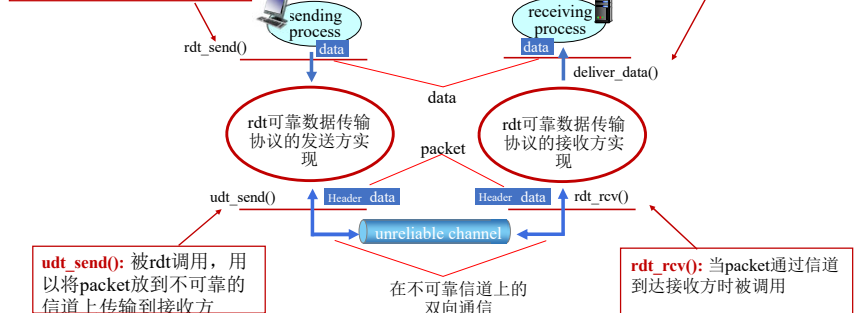
Transport Layer: 3-67

可靠数据传输原理 (rdt): 一些层间接口

2025中科大网络

rdt_send(): 被上层调用(e.g.,
 by app), 传给接收方应用的
 data通过层间接口交给下层

deliver_data(): 被rdt调用,
 将data交付给上层

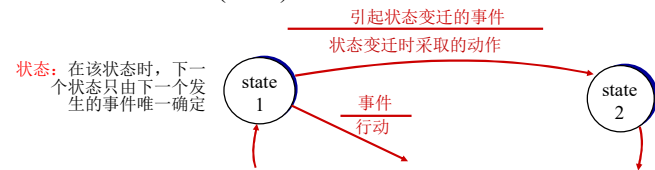


Transport Layer: 3-68

可靠数据传输原理: 开始

我们将会:

- 渐进地发展发送方和接收方的 **r**eliable **d**ata **t**ransfer (rdt)RDT协议
- 仅仅考虑单向的数据传输
 - 但是控制信息需要双向传输!
 - 双向的数据传输问题实际上是2个单向数据传输问题的综合
- 使用有限状态机 (FSM) 来描述发送方和接收方



Transport Layer: 3-69

rdt1.0: 在可靠信道上的可靠数据传输

- 下层的信道是完全可靠的
 - 没有比特出错
 - 没有分组丢失
- 发送方和接收方的 **相互独立的 (不需要交互控制信息)** FSMs:



Transport Layer: 3-70

Rdt2.0: 具有比特差错的信道

- 下层信道可能会出错: 噪声将分组中的比特翻转
 - 用 **校验和** 来检测比特差错
- 问题: 如何从错误中恢复出来?

人类如何从对话中的“错误”中恢复过来?

Transport Layer: 3-71

Rdt2.0: 具有比特差错的信道

- 下层信道可能会出错: 将分组中的比特翻转
 - 用 **校验和** 来检测比特差错
- 问题是: 如何从错误中恢复出来?
 - **acknowledgements (ACKs)**: 接收方显式地告诉发送方分组已被正确接收
 - **negative acknowledgements (NAKs)**: 接收方显式地告诉发送方分组发生了差错
 - 发送方 **重传** NAK对应的分组

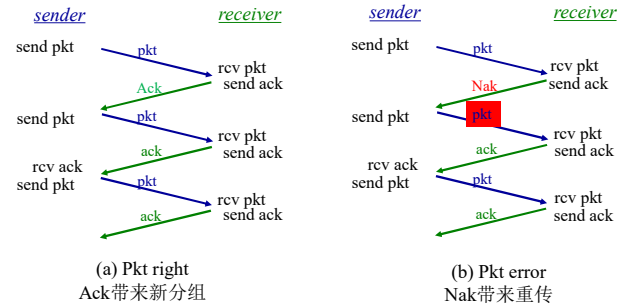
stop and wait

发送方每发送一个分组, 都要等待接收方给出回应, 然后传输下一个分组

Transport Layer: 3-72

rdt2.0的运行

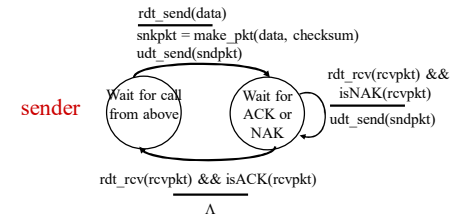
2025中科大网络



Transport Layer: 3-73

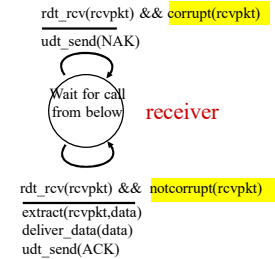
rdt2.0: FSM描述

2025中科大网络



- 缓冲区放置副本
- 计算校验和
- 根据ack和nak，决定发新的还是老的

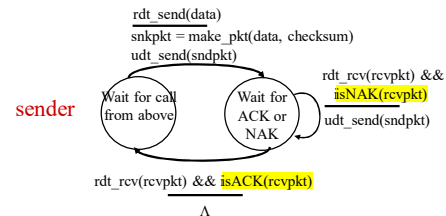
- 校验有无出错
- 从而发送ack或者nak，决定是否向上交付



Transport Layer: 3-74

rdt2.0: FSM描述

2025中科大网络

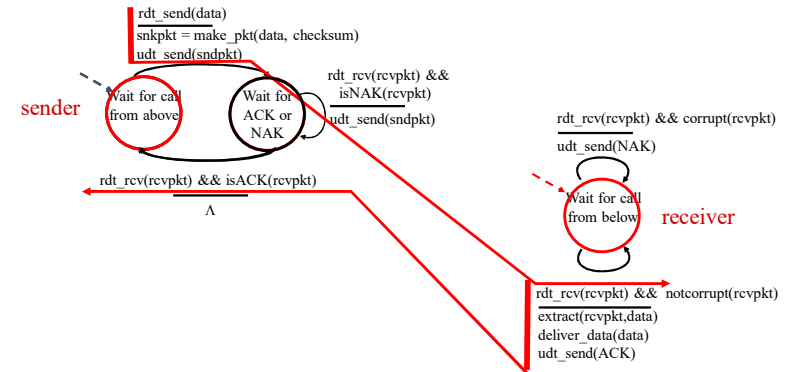


注：接收方的“状态”（接收方是否正确接收了我的消息？）对于发送方来说是未知的，除非接收方以某种方式与发送方进行了通告，这就是为什么需要一个协议的原因！交换控制信息（ACK/NAK），从而决定传新的还是重传



rdt2.0: 没有差错时的运行

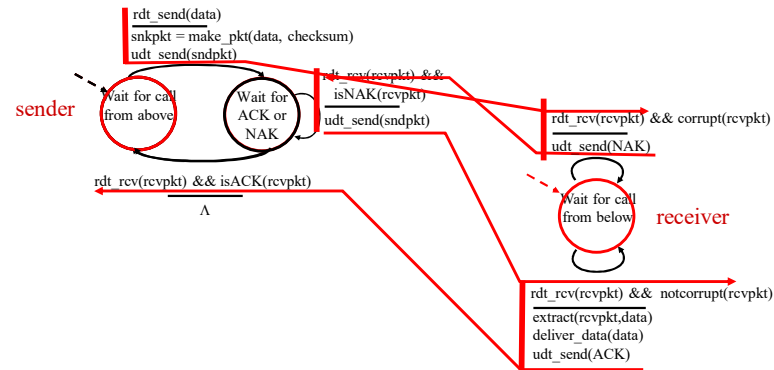
2025中科大网络



Transport Layer: 3-76

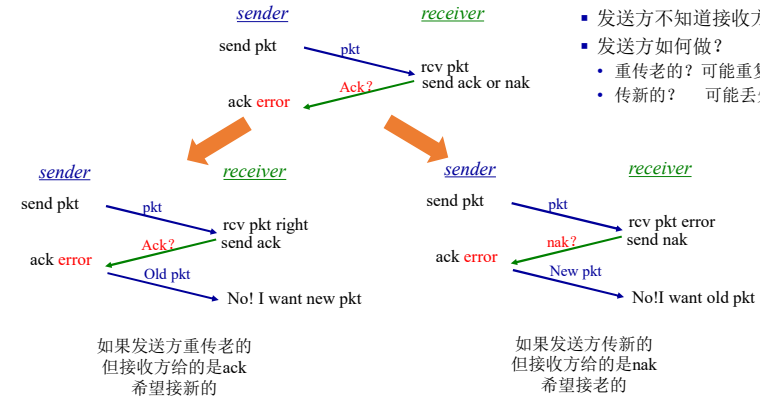
rdt2.0: 出错分组场景

2025中科大网络



rdt2.0的两难境地

2025中科大网络



Rdt2.0 ack出错问题的解决

2025中科大网络

意味着rdt 2.0存在致命缺陷

- 引入新的机制：发送方在每个分组上编上序号
- 发送方的措施：当ACK/NAK出错时，发送方重传当前的分组

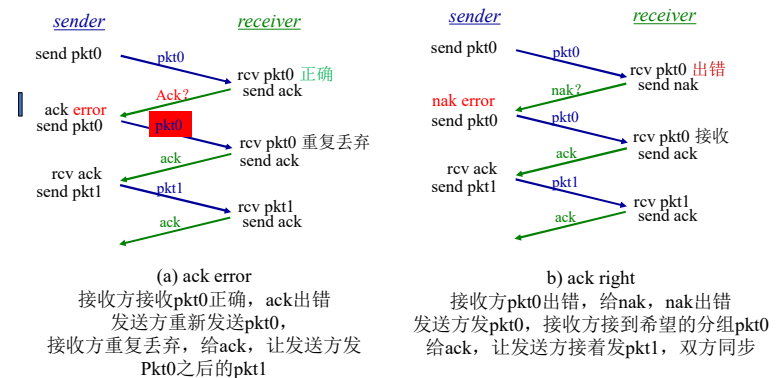
序号引入的重复问题：

- 可能出现重复
- 通过序号，接收方可以判断重复
 - 重复：丢弃（并不上交给用户），给ack，让发送方发送重复之后的新的
 - 不重复：上交给用户，给ack，让发送方接着发送更加新的

Transport Layer: 3-79

rdt2.1的运行

2025中科大网络



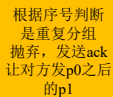
2025中科大网络



2025中科大网络



2025中科大网络



发送方收到错误的
Ack，重放老分组
它甚至都不知道
这个分组是ack

(c) ack error

Transport Layer: 3-83

2025中科大网络

接收方:

- 必须检测收到的分组是否重复？
 - 接收方的状态（用一个变量维护）会指示希望接收到的分组序号为0还是1
- 注意：接收方并不知道发送方是否
 - 没有安排确认的确认
 - 具体解释见后

Transport Layer: 3-84

rdt2.2: 无NAK的协议

- 取消NAK，只有ACK，功能同rdt2.1，但ACK要编号
- 作为NAK的替代，接收方给出对**最后被收到分组**的确认
 - 接收方必须显式地指明最后被确认分组的序号
- 发送方收到重复序号的ACK，动作如同收到NAK，重传当前分组
- 为后面的一次发送多个数据（流水线协议）单位做一个准备
 - 流水线协议一次能够发送多个，如每一个分组的应答都有：ACK，NACK；麻烦
 - 使用对前一个数据单位的ACK，代替本数据单位的nak，确认信息减少一半，协议处理简单

正如我们将要看到的，TCP使用这种方法，是无NAK的

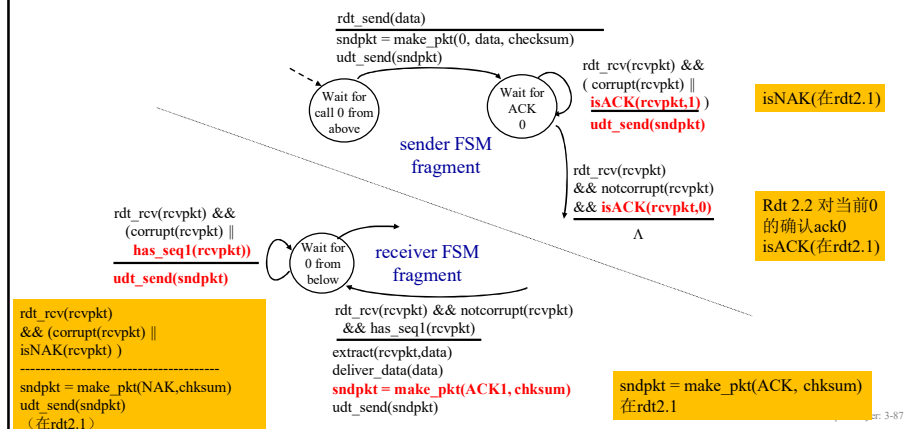
Transport Layer: 3-85

rdt2.2: 无NAK的协议



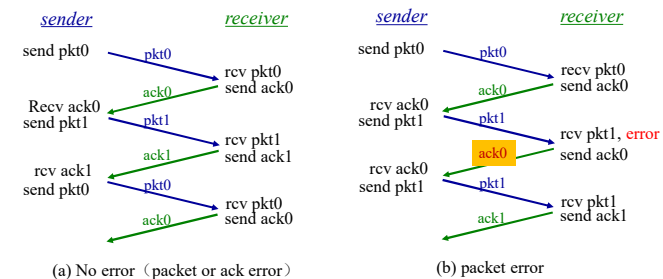
Transport Layer: 3-86

rdt2.2: 发送方,接收方的部分FSM



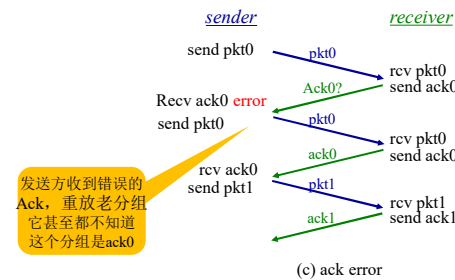
3-87

rdt2.2的运行



Transport Layer: 3-88

rdt2.2的运行



发送方收到错误的Ack，重放老分组
它甚至都不知道这个分组是ack0

根据序号判断是重复分组
抛弃，发送ack0
让对方发p0之后的p1

Transport Layer: 3-89

rdt3.0: 同时具有出错和丢失情况的信道

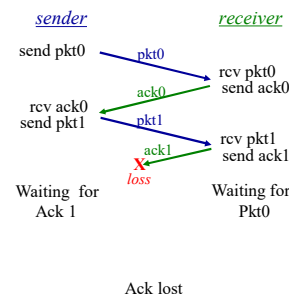
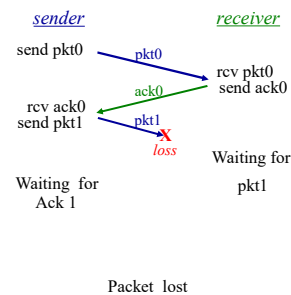
新的信道假设：下层信道可能丢失分组（数据分组和ack分组）

- 校验和+序号+ ack+ 重传机制等可能有助于此
- 但还是不够完备

问：人类如何处理会话中发送方到接收方丢失的内容？

Transport Layer: 3-90

rdt2.2的问题：丢失会造成死锁



Transport Layer: 3-91

rdt3.0: 同时具有出错和丢失情况的信道

方法：发送方等待ACK一段合理的时间

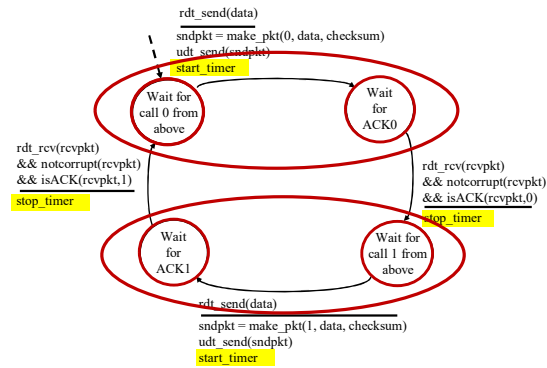
- 发送端超时重传：如果到时没有收到ACK->重传
- 问题：如果分组（或ACK）只是被延迟了，而不是丢失了：
 - 重传将会导致数据重复，但利用序列号已经可以处理重复问题！
 - 接收方必须（在ack中）指明被正确接收的序列号
- 当“合理的”等待时间到时之后，需要一个倒数定时器来产生中断



Transport Layer: 3-92

rdt3.0 发送方

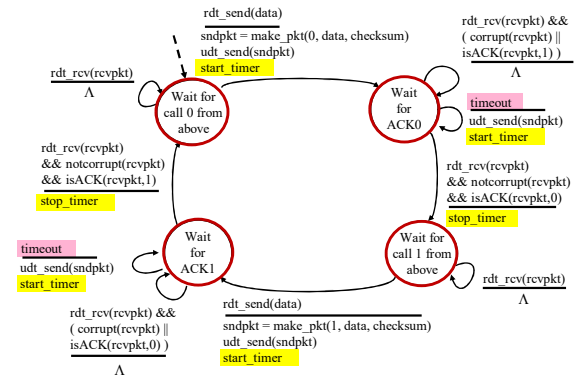
2025中科大网络



Transport Layer: 3-93

rdt3.0 发送方

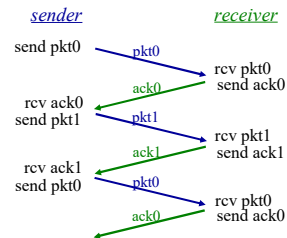
2025中科大网络



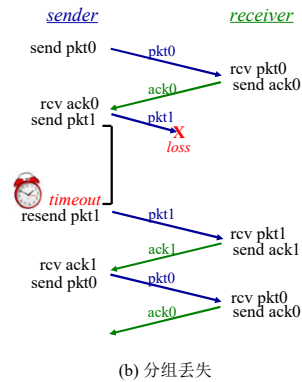
Transport Layer: 3-94

rdt3.0运行实例

2025中科大网络



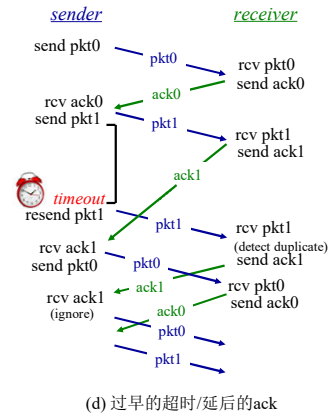
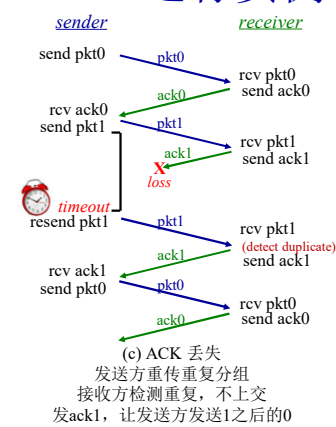
(a) 没有丢失



(b) 分组丢失

rdt3.0运行实例

2025中科大网络



- 过早超时（延迟的ACK）也能够正常工作；但是效率较低，一半的分组和确认是重复的；
- 设置一个合理的超时时间也是比较重要的；

Transport Layer: 3-96

rdt3.0的性能(停止等待协议)

2025中科大网络

▪ U_{sender} : **utilization 利用率** – 发送方忙于有效发送所占的时间比例

▪ 例子: 1 Gbps链路, 15 ms 传播延迟, 8000 bit 分组

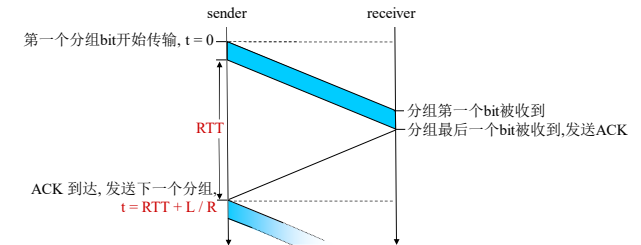
- 将分组传输到信道的时间:

$$D_{\text{trans}} = \frac{L}{R} = \frac{8000 \text{ bits}}{10^9 \text{ bits/sec}} = 8 \text{ microseconds}$$

Transport Layer: 3-97

rdt3.0: 停止等待运行

2025中科大网络



Transport Layer: 3-98

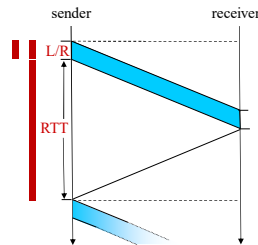
rdt3.0: 停止等待运行

2025中科大网络

$$U_{\text{sender}} = \frac{L/R}{\text{RTT} + L/R}$$

$$= \frac{.008}{30.008}$$

$$= 0.00027$$

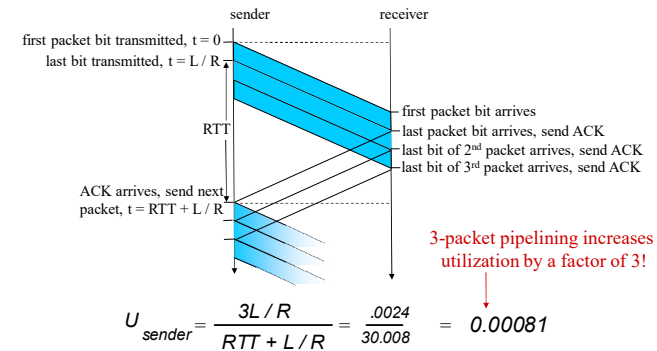


- rdt 3.0 协议性能受限!
- 协议本身限制了下层信道的使用效率

Transport Layer: 3-99

流水线协议: 增加利用率

2025中科大网络



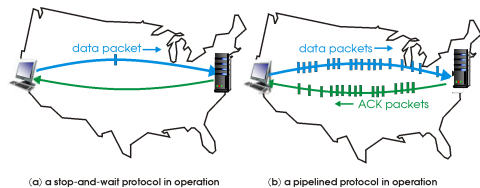
Transport Layer: 3-100

rdt3.0改进: 流水线协议

2025中科大网络

流水线:发送方允许多个、等待确认的“飞行”分组

- 必须增加序号的范围: 用多个bit表示分组的序号
- 在发送方/接收方要有缓冲区
 - 发送方缓冲: 存储尚未得到确认的分组, 因为可能需要重传;
 - 接收方缓冲: 上层用户取用数据的速率≠接收到的数据速率; 接收到的数据可能乱序, 排序交付 (可靠)



Transport Layer: 3-101

通用: 滑动窗口(slide window)协议

2025中科大网络

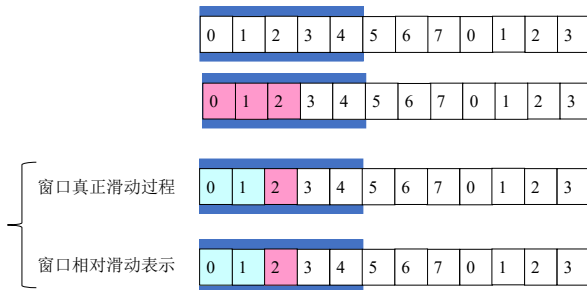
- 发送缓冲区**
 - 形式: 内存中的一个区域, 落入缓冲区的分组可以发送
 - 功能: 用于存放已发送, 但是没有得到确认的分组
 - 必要性: 需要重发时可用
- 发送缓冲区中的分组**
 - 未发送的:** 落入发送缓冲区的分组, 可以立即连续发送出去;
 - 已经发送出去、等待对方确认的分组:** 发送缓冲区中的分组只有得到接收方的确认才能删除
- 发送缓冲区的大小: 一次最多可以发送多少个未经确认的分组**
 - 停止等待协议=1
 - 流水线协议>1
 - 合理的值, 不能很大
 - 链路利用率不可能超过100%

Transport Layer: 3-102

发送窗口滑动过程-相对表示方法

2025中科大网络

- 采用相对移动方式表示, 分组不动
- 可缓冲范围移动, 代表一段可以发送的权力

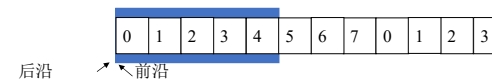


Transport Layer: 3-103

滑动窗口(slide window)协议

2025中科大网络

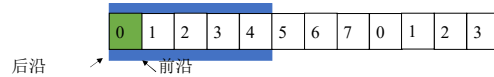
- 发送窗口:** 发送缓冲区内容的一个范围
 - 那些已发送但是未经确认分组的序号构成的空间
- 发送窗口尺寸** ≤ 发送缓冲区尺寸
- 一开始: 没有发送任何一个分组, 发送窗口
 - 后沿=前沿
 - 之间为发送窗口, 其尺寸=0
- 每发送一个分组, 前沿前移一个单位



Transport Layer: 3-104

发送窗口的移动->前沿移动

- 发送窗口前沿移动的极限：不能够超过发送缓冲区



Transport Layer: 3-105

发送窗口的移动->后沿移动

- 发送窗口后沿移动
 - 条件：收到老分组的确认
 - 结果：后沿移动，发送缓冲区罩住新的分组，来了分组可以发送
 - 移动的极限：不能够超过前沿（不能够确认尚未发送的分组）



Transport Layer: 3-106

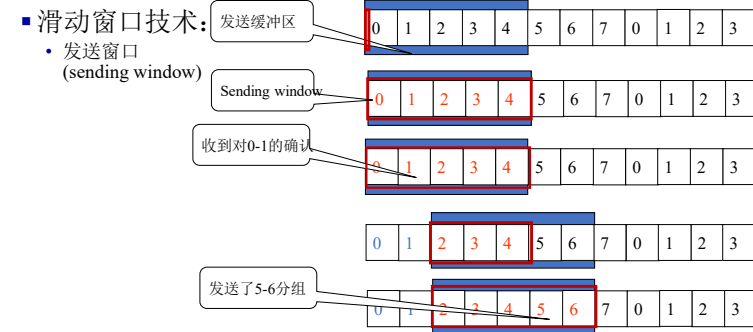
发送窗口的移动->后沿移动

- 发送窗口后沿移动
 - 条件：收到老分组(后沿)的确认
 - 结果：发送缓冲区罩住新的分组，来了分组可以发送
 - 移动的极限：不能够超过前沿（不能够确认尚未发送的分组）



Transport Layer: 3-107

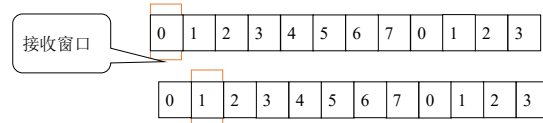
滑动窗口(slide window)协议



Transport Layer: 3-108

滑动窗口(slide window)协议-接收窗口

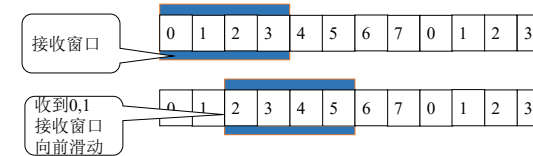
- 接收窗口 (receiving window)=接收缓冲区(假设分组到了上层可收)
 - 用于控制哪些分组可以接收;
 - 只有收到的分组序号落入接收窗口内才允许接收
 - 若序号在接收窗口之外, 则丢弃;
 - 窗口尺寸 $W_r=1$ 时, 则只能顺序接收;
 - 滑动条件: W_r 所在位置的分组到达
 - 例子: $W_r=1$, 在0的位置; 只有0号分组可以接收;
 - 向前滑动一个, 罩在1的位置, 如果来了第2号分组, 则丢弃;



Transport Layer: 3-109

滑动窗口(slide window)协议-接收窗口

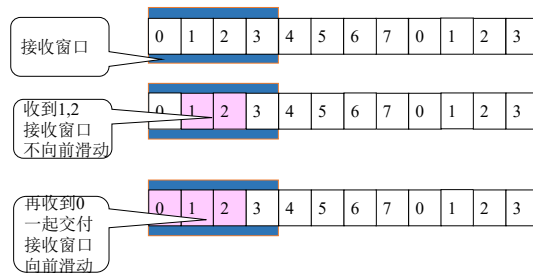
- 接收窗口的滑动和发送确认
 - 窗口尺寸 $W_r > 1$ 时, 则可以乱序接收
 - 但交付给上层的分组, 要按序
 - 滑动条件:
 - 低序号的分组到来, 向上交付, 接收窗口移动;



Transport Layer: 3-110

滑动窗口(slide window)协议-接收窗口

高序号分组乱序到, 缓存但不交付 (因为要实现 rdt, 不允许失序), 不滑动



Transport Layer: 3-111

接收窗口的大小决定了是GBN还是SR

- 接收窗口尺寸=1: 只能顺序接收, 乱序不收
 - 接收方给出连续收到的最大分组确认, 决定了必须**累计确认**
 - 发送方知道乱序的分组接收方不收, 只设1个超时定时器与最老的分组关联
 - 一旦**超时 go-back-n**, 重发发送窗口**所有**已发送待确认的分组,
 - 发送方判断: 超时的分组接收方没收到, 对于超时分组后面的已发送分组, 接收方即使收到了也会抛弃

Transport Layer: 3-112

接收窗口的大小决定了是GBN还是SR

- 接收窗口尺寸 >1 ：可以乱序接收
 - 接收方单独给出收到分组的确认，决定了可以**每个分组独立确认**
 - 发送方知道接收方可以乱序接收，为每个分组设置一个独立计时器
 - 哪个分组到来了，发送方**选择性重发SR**那个分组
 - 而不是重发发送窗口的所有分组
 - 超时分组后面的分组接收方（接收缓冲区大于1）可能已经缓存，未见得需要重发

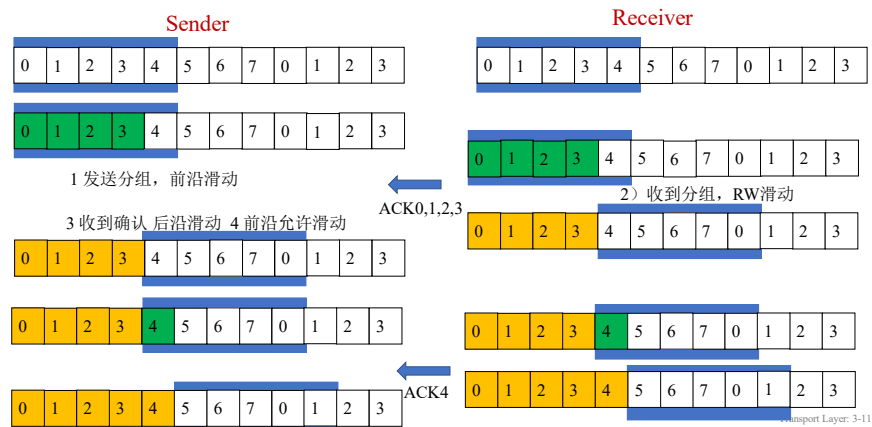
Transport Layer: 3-113

正常情况下的2个窗口互动

- 发送窗口
 - 有新的分组落入发送缓冲区范围，发送 $\rightarrow$ SW 前沿滑动
 - 来了老低序号分组的确认 $\rightarrow$ 后沿向前滑动 $\rightarrow$ 新分组可以落入发送缓冲区范围
- 接收窗口
 - 收到分组，落入到接收窗口范围内，接收
 - 是低序号，发送确认给对方，RW 滑动
- 发送端上面来了分组 $\rightarrow$ 发送窗口滑动 $\rightarrow$ 接收窗口滑动 $\rightarrow$ 发确认

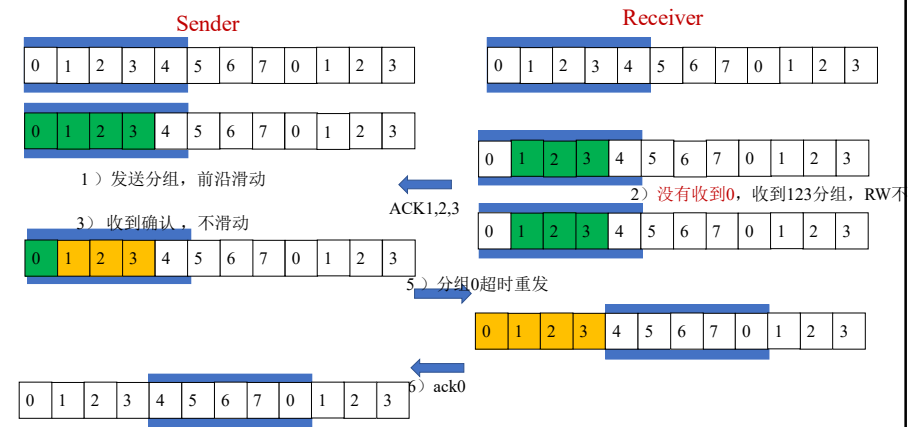
Transport Layer: 3-114

正常情况下的2个窗口互动

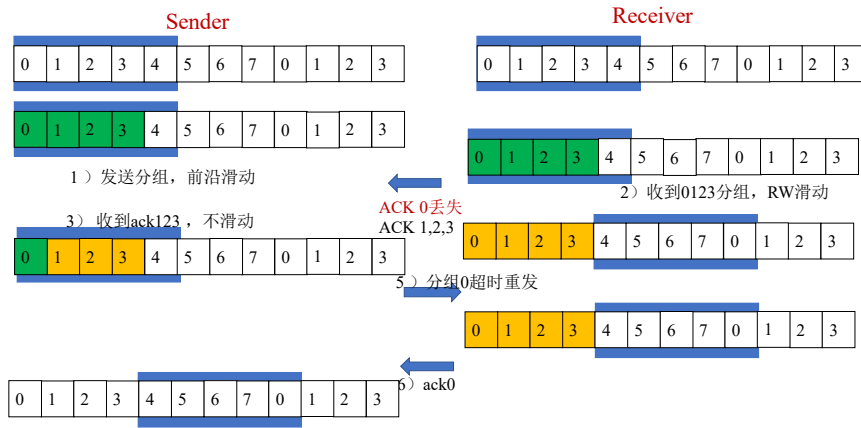


Transport Layer: 3-115

异常情况下的2个窗口互动（分组出问题）



异常情况下的2个窗口互动 (ack出问题)



异常情况下GBN的2窗口互动

- 发送窗口
 - 新分组落入发送缓冲区范围, 发送->前沿滑动
 - 超时重发机制让发送端将发送窗口中的所有分组发送出去
 - 来了老分组的重复确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围 (此时如果发送缓冲区有新的分组可以发送)
- 接收窗口
 - 收到乱序分组, 没有落到接收窗口范围内, 抛弃
 - 如信道使得中间分组丢失, (重复) 发送老分组的确认, 累计确认

Transport Layer: 3-118

异常情况下SR的2窗口互动

- 发送窗口
 - 新分组落入发送缓冲区范围, 发送->前沿滑动
 - 超时重发机制让发送端将超时的分组重新发送出去
 - 来了乱序分组的确认->后沿不向前滑动->新的分组无法落入发送缓冲区的范围 (此时如果发送缓冲区有新的分组可以发送)
- 接收窗口
 - 收到乱序分组, 落入到接收窗口范围内, 接收
 - 如信道使得中间分组丢失, 发送正确分组的单独确认

Transport Layer: 3-119

GBN协议和SR协议的表现异同

- 相同之处
 - 发送窗口>1
 - 一次能够可发送多个未经确认的分组
- 不同之处
 - GBN: 接收窗口尺寸=1
 - 接收端: 只能顺序接收, 确认是累计确认
 - 发送端: 从表现来看, 一旦一个分组没有发成功, 如: 0,1,2,3,4; 假如1未成功, 2,3,4都发送出去了, 要返回1再发送; GBN (发送方知道接收方肯定不收2,3,4)
 - SR: 接收窗口尺寸>1
 - 接收端: 可以乱序接收, 确认是选择性确认
 - 发送端: 发送0,1,2,3,4, 一旦1超时 (未成功), 2,3,4已发送, 无需重发所有已发, 只选择性重新发送1 (发送方知道接收方可能已缓存)

http://wps.pearsoned.com/ecs_kurose_compnw_6

Transport Layer: 3-120

流水线协议总结：GBN和SR的机制异同

Go-back-N:

- 发送端最多在流水线中有N个未确认的分组
- 接收端只是发送**累计型确认**
 - 接收端如果发现gap, 不确认新到来分组, 而是确认顺序到达的最大分组
- 定时器和发送端最老的未确认分组关联
 - 只需设置**一个定时器**
 - 当定时器到时, **重传所有未确认分组**

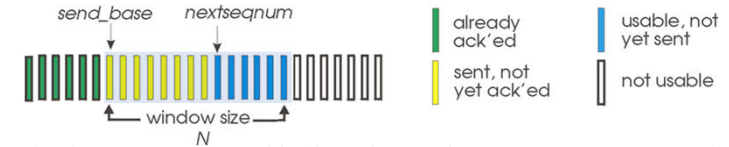
Selective Repeat:

- 发送端最多在流水线中有N个未确认的分组
- 接收方对每个到来的分组**单独确认**
- 发送方为每个**未确认的分组保持一个定时器**
 - 当超时定时器到时, 只是选择性重发超时的未确认分组

Transport Layer: 3-121

Go-Back-N: 发送方

- 发送方: 窗口最大值N, 那些已经发送但是未确认的分组
 - 分组头部用kbit表示分组序号



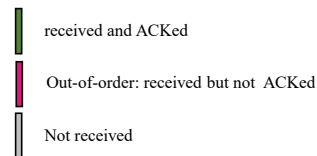
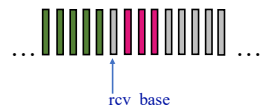
- **累积性确认**: ACK(n): 知道接收方确认所有直到n (包括n) 的所有分组
 - 当接收到ACK(n): 将窗口 (后沿) 移动到 $n+1$, $n+1$ 分组是最老待确认的分组
- 唯一的一个超时定时器, 和最老的已发送待确认分组关联
- **timeout(n)**: 重新传送从n开始的所有窗口中的分组, 重新发送所有已经发送尚未确认的分组

Transport Layer: 3-122

Go-Back-N: 接收方

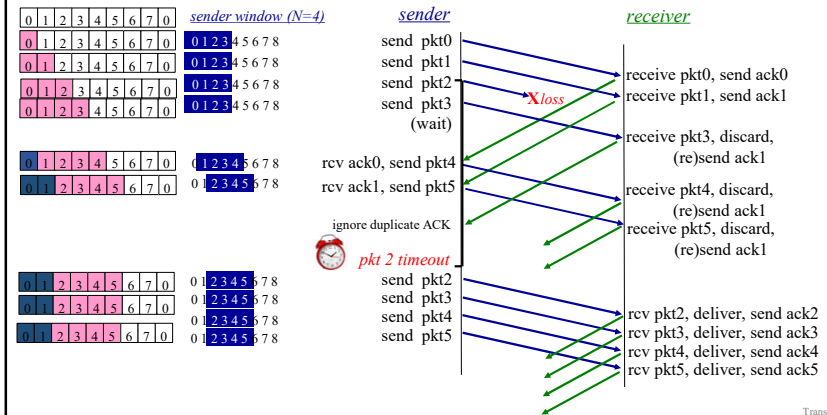
- ACK: 只对到目前为止**顺序收到**的序号最高**正确**分组的确认
 - 可能会产生重复的ACKs
 - 只需要记住rcv_base
- 如果收到了乱序到达的分组:
 - 丢弃该分组 (而不是缓存)
 - **重新发送确认**: 对顺序到达的具有最高序号的分组做确认

接收者序号空间视图:



Transport Layer: 3-123

Go-Back-N运行实例



Transport Layer: 3-124

2025中科大网络



2025中科大网络

- Transport Layer: 3-126

2025中科大网络



2025中科大网络

「接收方

packet n in $[\text{rcvbase}, \text{rcvbase}+N-1]$

- 发送 ACK(n)
- 乱序到达: 缓存
- 顺序到达: 交付给上层 (同时也交付哪些后面的已顺序到达的缓存分组), 移动窗口到下一个仍然未接收到的分组位置上

packet n in $[rcvbase-N,rcvbase-1]$

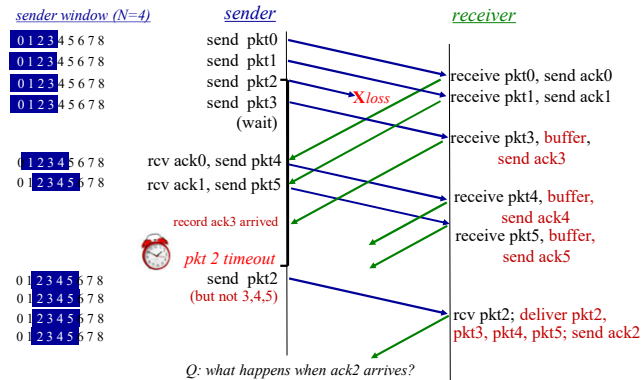
- ACK(n)

其他:

- 忽略

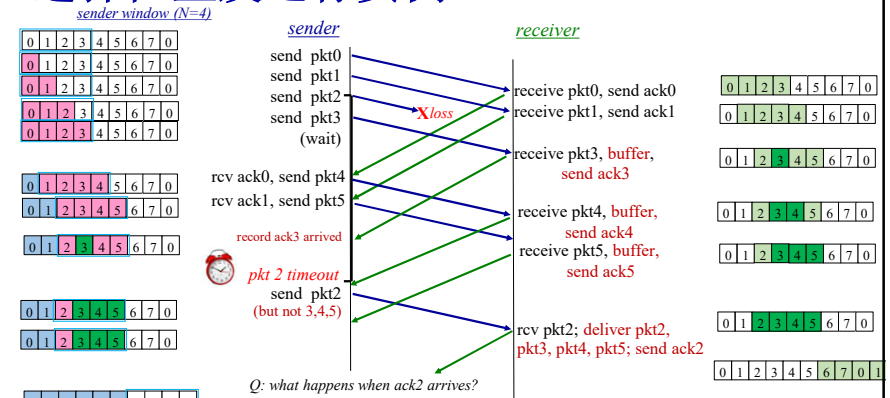
选择性重发运行实例

2025中科大网络



选择性重发运行实例

2025中科大网络



GBN和SR适用范围对比

2025中科大网络

	GBN	SR
优点	简单, 所需资源少 (接收方一个缓存单元)	出错时重传一个代价小
缺点	一旦出错, 回退N步代价大	复杂, 所需要资源多 (接收方多个缓存单元)

适用范围

- 出错率低: 比较适合GBN, 出错非常罕见, 没有必要用复杂的SR, 为罕见的事件做日常的准备和复杂处理
 - 带宽延迟积小, 或出错率小
- 链路容量大 (延迟大、带宽大): 比较适合SR而不是GBN, 一点出错如果回退N步代价太大
 - 带宽延迟积大, 且 (或?) 出错率高

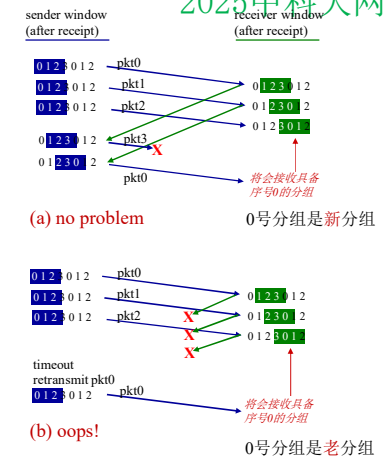
Transport Layer: 3-131

选择性重发的一个两难境地

2025中科大网络

实例:

- seq #: 0, 1, 2, 3 (base 4 counting)
- 窗口大小为3

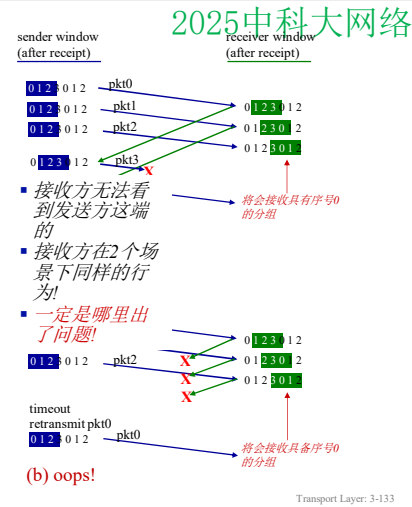


选择性重发： 两难境地！

例子：

- seq #s: 0, 1, 2, 3 (base 4 counting)
- 窗口大小=3

Q: 为了避免场景（b）中的问题，序列#大小和窗口大小之间需要什么关系？



传输层：提纲

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
 - 段结构
 - 可靠数据传输
 - 流量控制
 - 连接管理
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



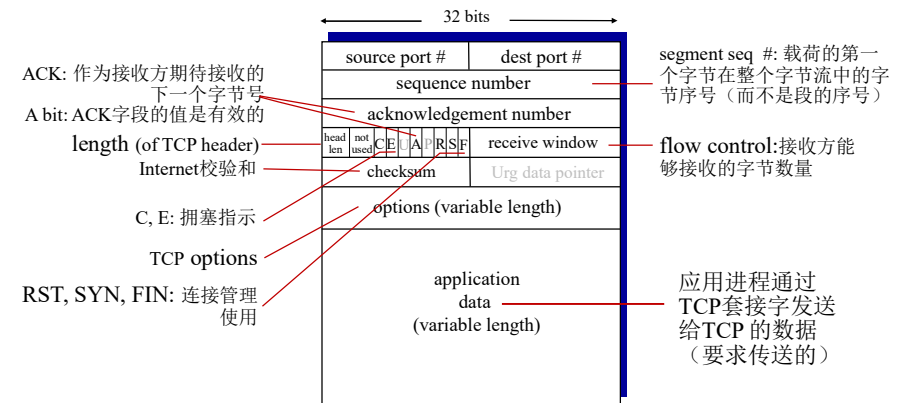
Transport Layer: 3-134

TCP: 概述 RFCs: 793, 1122, 2018, 5681, 7323

- 点到点：
 - 一个发送方，一个接收方
- 可靠的，保序的字节流：
 - 不提供“应用报文之间的边界”信息
- 全双工数据通信：
 - 在一个TCP连接上双向数据通信
 - MSS: 按照最大报文段进行应用报文的切分
- 累积性确认（类GBN）
- 流水线：
 - TCP 拥塞控制和流量控制协同设置发送窗口大小
- 面向连接：
 - 在数据传输前，发送方和接收方通过握手初始化（交换一些控制信息）发送方和接收方的状态
- 流量控制和拥塞控制：
 - 发送方不会淹没接收方和网络



TCP 段结构



TCP 序号, 确认号

2025中科大网络

序号:

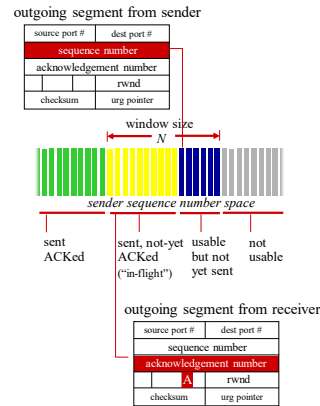
- 段数据的第一个字节在整个字节流中的位置 (以字节计)

确认:

- 作为接收方期待对端从该字节开始传输
- 累积性确认

Q: 接收方如何处理乱序到达的段?

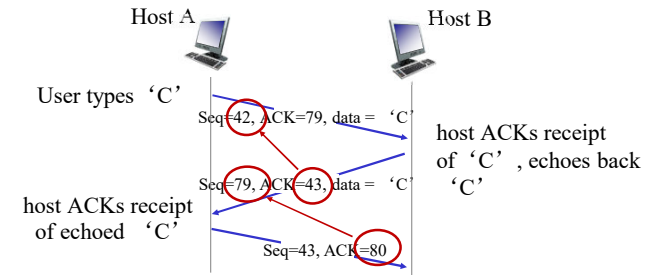
- 4: 协议规范没有规定, 取决于实现者



Transport Layer: 3-137

TCP 序号, 确认号

2025中科大网络



simple telnet scenario

Transport Layer: 3-138

TCP 往返时间, 超时定时器设置

2025中科大网络

Q: TCP如何设置超时时间?

- 比RTT要长, 但是RTT本身是变化的!
- 太短: 过早超时, 没有必要的重传
- 太长: 对于段的丢失恢复反应较慢

Q: 如何估计 RTT?

- SampleRTT: 测量从段发出到确认回来之间的时间
 - 忽略重传
- SampleRTT 变化非常大, 需要估计 RTT “平滑一些”
 - 对最近的若干测量值取平均, 而不仅仅是最近的一个采样 SampleRTT

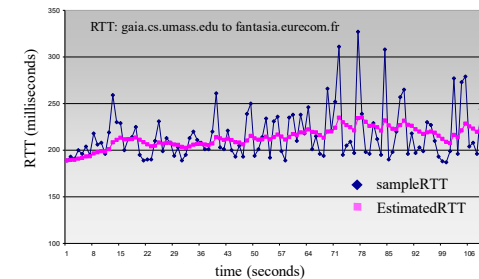
Transport Layer: 3-139

TCP 往返时间, 超时定时器设置

2025中科大网络

$$\text{EstimatedRTT} = (1 - \alpha) * \text{EstimatedRTT} + \alpha * \text{SampleRTT}$$

- exponential weighted moving average (EWMA): 指数加权移动平均
- 过去的采样值对于平均值的影响呈指数规律快速下降
- 典型值: $\alpha = 0.125$



Transport Layer: 3-140

TCP 发送方 (简化版)

- 重传后沿最老的报文段（似SR）
 - 重新启动定时器
- 事件：收到ACK**
- 如果是对尚未确认的报文段确认
 - 更新已被确认的报文序号
 - 如果当前还有未被确认的报文段，重新启动定时器

safety margin

- $$\text{DevRTT} = (1-\beta) * \text{DevRTT} + \beta * |\text{SampleRTT} - \text{EstimatedRTT}|$$

* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Transport Layer: 3-141

Transport Layer: 3-142

2025中科大网络

TCP 接收方动作

延迟的ACK。对另一个按序报文段的到达最多等待500ms。如果下一个报文段在这个时间间隔内没有到达，则发送一个ACK

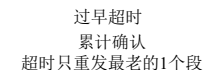
立即发送单个累积ACK，以确认两个按序报文段。

立即发送**重复ACK**，指明下一个期待字节的序号

若该报文段起始于间隔（gap）的低端，
则立即发送ACK。

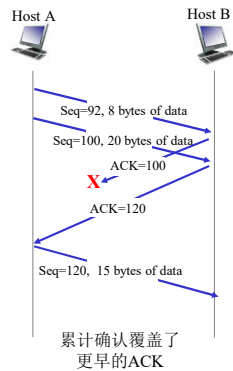
Transport Layer: 3-143

2025中科大网络



Transport Layer: 3-144

TCP: 重传场景



Transport Layer: 3-145

TCP 快速重传

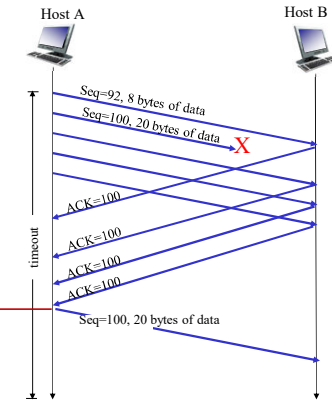
TCP 快速重传

如果发送方收到对某个段的3个冗余ack，重新传送具有最小序号的段，即使其还未超时

- 大概率未确认的段丢失了，虽未超时，仍然快速重发，提高恢复的速度



收到三个重复ACK表示：ACK对应的段没有被接收方收到，但是该段之后的三个段都被收到了——很有可能该段丢失了。所以需要重新发送！



Transport Layer: 3-146

传输层：提纲

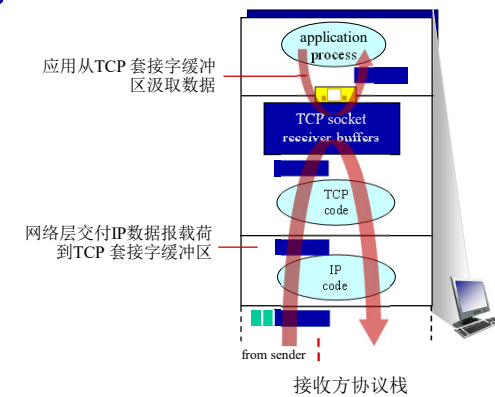
- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
 - 段结构
 - 可靠数据传输
 - 流量控制
 - 连接管理
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



Transport Layer: 3-147

TCP 流量控制

Q: 当网络层提交数据的速率比应用进程从socket缓冲区中汲取的速率要来得快，会发生什么？



Transport Layer: 3-148

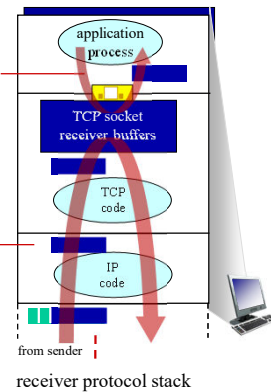
TCP 流量控制

Q: 当网络层提交数据的速率比应用进程从socket缓冲区中汲取的速率要来得快, 会发生什么?



应用从TCP 套接字缓冲区汲取数据

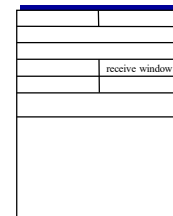
网络层交付IP数据报载荷到TCP 套接字缓冲区



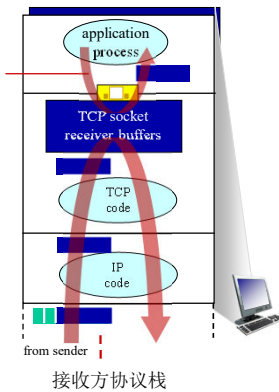
Transport Layer: 3-149

TCP 流量控制

Q: 当网络层提交数据的速率比应用层从socket缓冲区中汲取的速率要来得快, 会发生什么?



应用从TCP 套接字缓冲区汲取数据



Transport Layer: 3-150

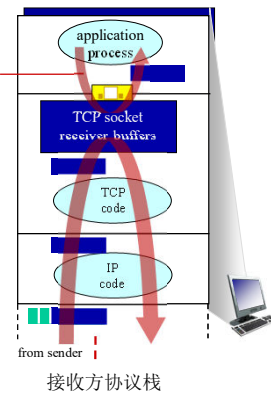
TCP 流量控制

Q: 当网络层提交数据的速率比应用层从socket缓冲区中汲取的速率要来得快, 会发生什么?

流量控制

接收方控制发送方, 这样发送方就不会发送的太快、太多, 以至于淹没接收方的缓冲区

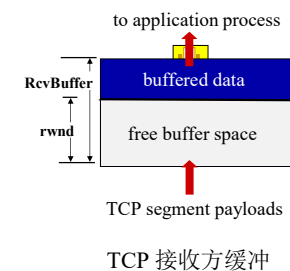
进程从TCP 套接字缓冲区汲取数据



Transport Layer: 3-151

TCP 流量控制

- TCP接收方用TCP头部的rwnd字段, 通告其可用缓冲空间
 - RcvBuffer大小通过socket选项设置 (典型值为4096字节)
 - 很多OS自动调整RcvBuffer的值
- 发送方限制待确认的(“in-flight”)数据在rwnd以下
- 确保接收方缓冲不会溢出

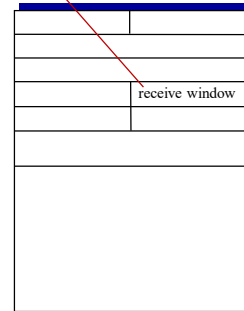


Transport Layer: 3-152

TCP 流量控制

- TCP接收方用TCP头部的rwnd字段，通告其可用缓冲空间
 - RcvBuffer大小通过socket选项设置（典型值为4096字节）
 - 很多OS自动调整RcvBuffer的值
- 发送方限制未确认的（“飞行中”）数据在rwnd以下
- 确保接收方缓冲不会溢出

flow control: 接收方愿意接收的字节数



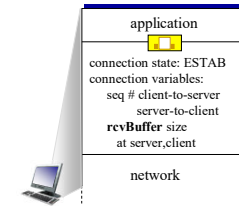
TCP segment format

Transport Layer: 3-153

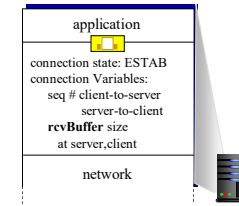
TCP 连接管理

在数据交换之前，发送方和接收方“握手”：

- 同意建立连接（每一方都知道对方同意建立连接）
- 确认该TCP连接的一些参数（例如：初始字节序号、接收窗口大小等）



Socket clientSocket =
newSocket("hostname", "port number");

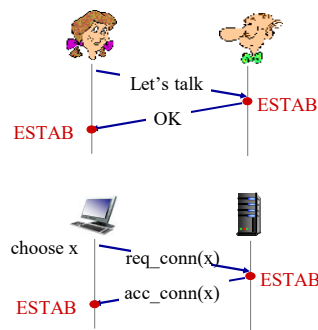


Socket connectionSocket =
welcomeSocket.accept();

Transport Layer: 3-154

同意建立连接

2次握手

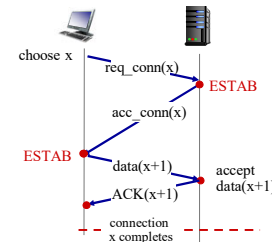


Q: 在网络中2次握手总是可行的吗？

- TCP段所在的分组延迟变化
- 重传报文（如：req_conn(x)）由于报文丢失
- 报文乱序
- 看不到对方（只能通过网络收到的对方信息来判断对方的状态）

Transport Layer: 3-155

2次握手场景

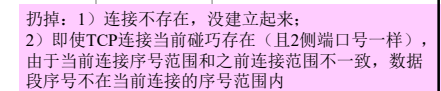


正常情况下没问题！



Transport Layer: 3-156

2025中科大网络



人类3次握手协议

2025中科大网络



Transport Layer: 3-161

关闭TCP连接

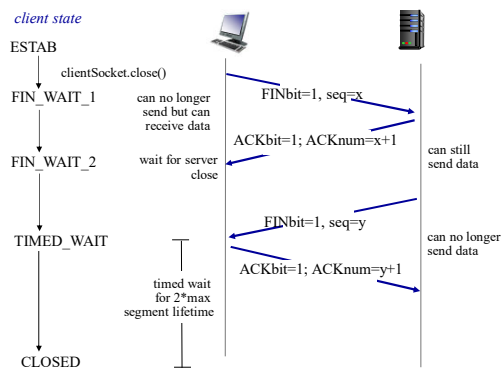
2025中科大网络

- **对称释放**：客户端和服务端每一方关闭各自一侧的连接
- 关闭连接的时序
 - 发送具备FIN位=1的TCP段
 - 收到FIN段之后，用ACK回应（一旦收到FIN，ACK可以和自己发出的连接释放FIN段进行合并）
- 两侧可以同时释放连接
- **不完美**，存在两军问题

Transport Layer: 3-162

关闭TCP连接

2025中科大网络



Transport Layer: 3-163

传输层：提纲

2025中科大网络

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- **拥塞控制原理**
- TCP拥塞控制
- 传输层协议的演化



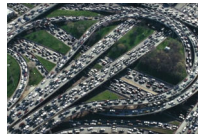
Transport Layer: 3-164

拥塞控制原理

2025中科大网络

拥塞:

- 非正式的定义: “太多的数据需要网络传输, 超过了网络 (而非主机) 的处理能力”
- 拥塞的表现:
 - 分组经历比较长的延迟(在路由器的队列中排队)
 - 分组丢失(路由器缓冲区溢出)
- 与流量控制不同
- a top-10 problem!



拥塞控制:

太多发送方, 发送的太快了



流量控制: 一个发送方向一个接收方发送的太快了

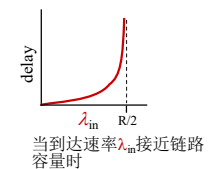
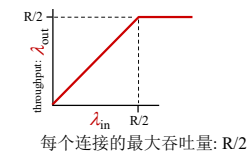
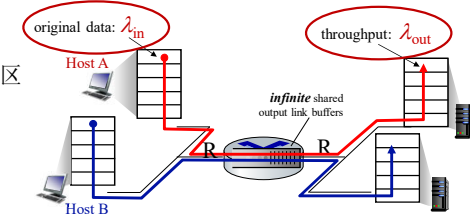
Transport Layer: 3-165

拥塞的原因/代价: 场景1

2025中科大网络

简单场景:

- 一个路由器, 无限大缓冲区
- 输入和输出链路容量: R
- 2个流
- 无需重传



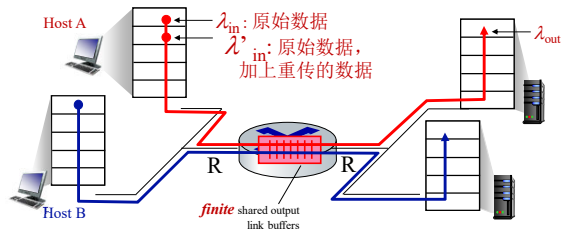
Q: 如果到达速率 λ_{in} 接近 $R/2$ 会如何?

Transport Layer: 3-166

拥塞的原因/代价: 场景2

2025中科大网络

- 一个路由器, 有限的缓冲
- 发送端重传丢失 (超时) 的分组
 - 应用层输入=应用层输出: $\lambda_{in} = \lambda_{out}$
 - 传输层输入包括重传: $\lambda'_{in} \geq \lambda_{in}$



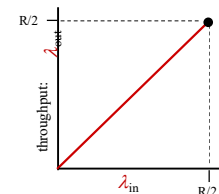
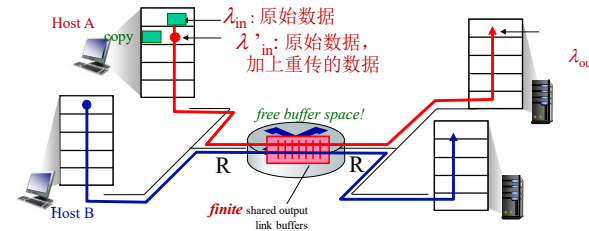
Transport Layer: 3-167

拥塞的原因/代价: 场景2

2025中科大网络

理想化假设: 主机能够获得有关于网络的完美信息

- 发送方只在路由器缓冲区有空闲空间时发送



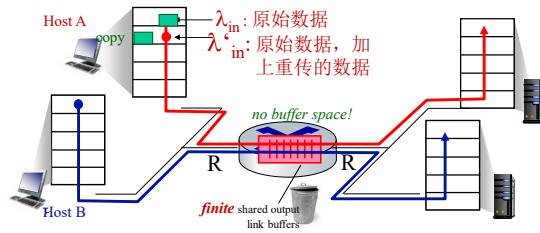
Transport Layer: 3-168

congestion的原因/代价: 场景2

2025中科大网络

理想化假设: 主机能够获得有关网络的**部分**完美信息

- 分组可能丢失（在路由器被抛弃）由于缓冲区溢出
- 发送方知道什么时候分组被抛弃: 如果知道该分组丢失了, 则只重传该丢失的分组



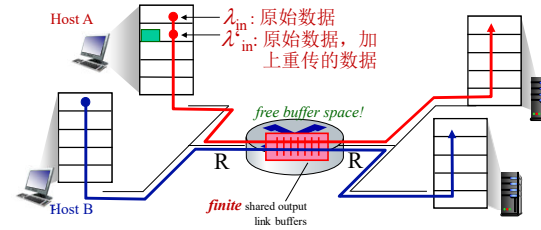
Transport Layer: 3-169

congestion的原因/代价: 场景2

2025中科大网络

理想化: 主机能够获得有关网络的**部分**完美信息

- 分组可能丢失（在路由器被抛弃）由于缓冲区溢出
- 发送方知道什么时候分组被抛弃: 如果知道该分组丢失了, 则只重传该丢失的分组



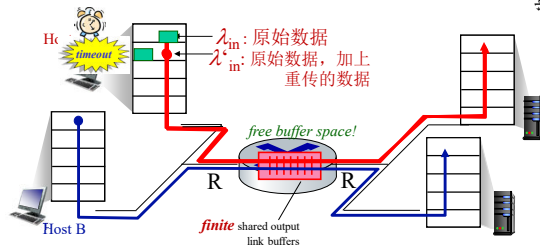
Transport Layer: 3-170

congestion的原因/代价: 场景2

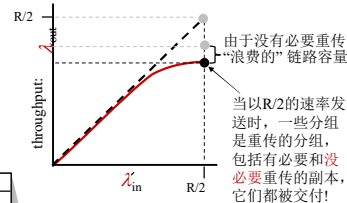
2025中科大网络

真实场景: **没有必要的重传**

- 分组可能丢失, 在路由器被丢弃, 由于缓冲区溢出-确实需要重传
- 但是可能在拥塞时发送方的定时器设置比较早, 发送了2个副本, **都**到达了接收方



Transport Layer: 3-171

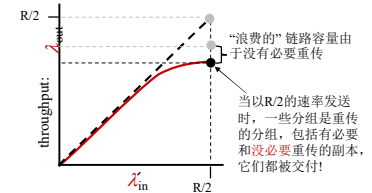


congestion的原因/代价: 场景2

2025中科大网络

真实场景: **没有必要的重传**

- 分组可能丢失, 在路由器被丢弃, 由于缓冲区溢出-确实需要重传
- 但是发送方的定时器设置比较早, 发送了2个副本, **都**到达了接收方



拥塞的代价:

- 为了在接收端达到一个有效输出, 网络需要做更多的工作 (重传)
- 包括那些没有必要的重传, 链路中包括了分组的多个副本
 - 进一步降低了最大可达到的吞吐量

Transport Layer: 3-172

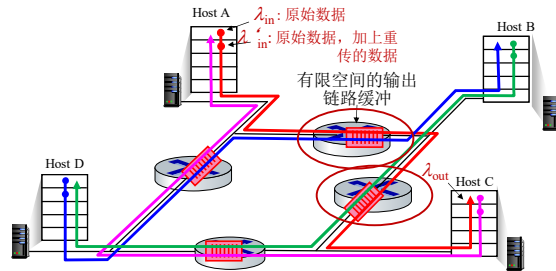
congestion的原因/代价: 场景3

2025中科大网络

- 4个发送端
- 多跳路径
- 超时/重传

Q: 当 λ 和 λ_{in}' 增大时会如何?

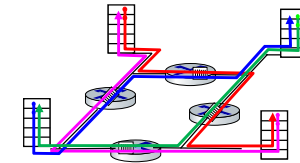
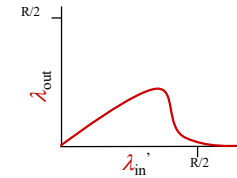
A: 当红色的 λ_{in}' 增大时, 所有到达上面队列的蓝色分组会被丢弃掉, 蓝色吞吐量 $\rightarrow 0$



Transport Layer: 3-173

congestion的原因/代价: 场景3

2025中科大网络



另外一个 congestion的代价:

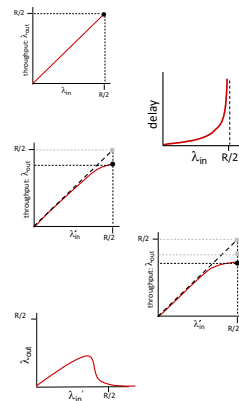
- 当分组被丢弃时, 作用于该分组的任何上游传输能力和使用过的缓存资源都被浪费了!

Transport Layer: 3-174

congestion的原因和代价: 一些洞察

2025中科大网络

- 吞吐不可能超过链路容量
- 当接近于链路容量时, 延迟增加
- 丢失/(有必要的)重传降低了有效吞吐量
- 没有必要的重传副本进一步减少有效吞吐量
- 对于下游被丢弃的分组, 其分组的任何上游传输能力/缓存都被浪费掉了



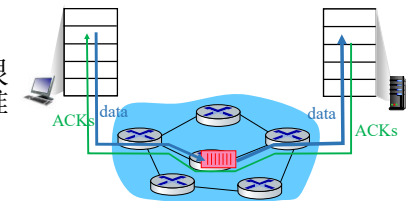
Transport Layer: 3-175

congestion控制的方法

2025中科大网络

端到端 congestion控制:

- 没有来自网络的显式反馈
- 是否 congestion的判断是由主机根据被观察到的延迟和丢失推测出来的
- TCP采用该方法

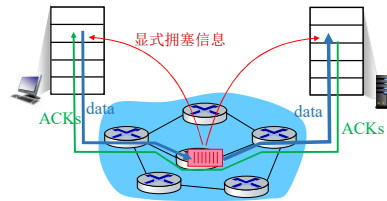


Transport Layer: 3-176

拥塞控制的方法

网络辅助的拥塞控制:

- 路由器在拥塞时，针对途径该路由器的发送方-接收方的数据流，向这些主机提供**直接的**拥塞反馈
- 可以在反馈信息中指示拥塞的水平和显式指定发送速率
- TCP ECN, ATM, DECbit 协议



Transport Layer: 3-177

传输层：提纲

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制**
- 传输层协议的演化



Transport Layer: 3-178

TCP 拥塞控制：总体机制

- 端到端的拥塞控制机制**
 - 路由器不向主机有关拥塞的反馈信息
 - 路由器的负担较轻
 - 符合网络核心简单的TCP/IP架构原则
 - 端系统根据自身得到的信息，判断是否发生拥塞，从而采取动作
- 与流量控制联合动作**
 - $\text{SendWin} = \min\{\text{cwnd}, \text{RcvWin}\}$
 - 同时满足流量控制和拥塞控制要求
 - 考虑拥塞控制时，通常将接收窗口设置成很大
 - 让流量控制问题不是瓶颈
 - 让拥塞控制问题暴露出来

Transport Layer: 3-179

拥塞控制的3个问题

- 如何检测拥塞：事件驱动**
 - 轻微拥塞**（3个冗余ACK）或 **拥塞**（超时）事件
 - 不发生以上情况：**不拥塞**（正常收到ack）
- 如何控制发送速率**
 - 控制cwnd窗口大小
 - 进而控速cwnd/RTT
- 控制策略：负反馈**
 - 在拥塞发送时如何动作，降低速率
 - 轻微拥塞，如何降低
 - 拥塞时，如何降低
 - 在不发生拥塞（缓解）时如何动作，增加速率

Transport Layer: 3-180

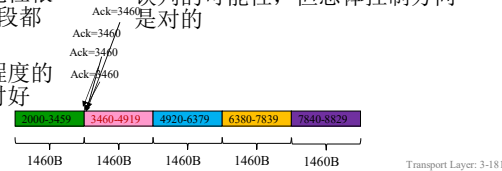
TCP CC问题1：拥塞感知

段的3个冗余ACK：轻微拥塞

- 第1个ack，正常，确认绿段，期待红段
- 第2个重复ack，意味着红段的后一段收到了，蓝段乱序到达
- 第2、3、4个ack重复->红段后第2、3、4个段收到了，橙段乱序到达，同时红段丢失的可能性很大（后面3个段都到了，红段都没到）
- 网络这时还能够进行一定程度的传输，拥塞但情况要比超时好

段超时了(丢失事件)：拥塞

- 超时时间到，段的确认没有来
- 原因1：网络拥塞（某路由器缓冲区没空间了，被丢弃）**概率大**
- 原因2：出错被丢弃（各级错误没有通过校验，被丢弃）**概率小**
- 一旦超时就认为拥塞了，有一定误判的可能性，但总体控制方向是对的



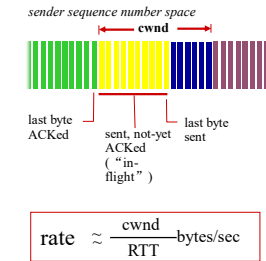
TCP CC问题2：速率控制方法

- 维持一个拥塞窗口的值：cwnd
- 发送端限制已发送但是未确认的数据量（的上限）：

$$\text{LastByteSent} - \text{LastByteAcked} \leq \text{cwnd}$$

- 从而粗略地控制发送方的往网络中注入的速率

- 通过负反馈，不精确的控制动作也可以达到目标



TCP 拥塞控制3：控制策略

状态	拥塞 (以下两种情况任何一种)		不拥塞 (正常收到确认)
SS (离拥塞还远)	3个冗余ACK	ssthresh=cwnd/2 cwnd=ssthresh+3 保持在SS	cwnd 乘性增
	超时	ssthresh=cwnd/2 cwnd=1 保持在SS	
CA (可能发送拥塞)	3个冗余ACK	ssthresh=cwnd/2 cwnd=ssthresh+3 保持CA	cwnd 线性增
	超时	ssthresh=cwnd/2 cwnd=1 进入SS	

183

TCP 拥塞控制3：控制策略

cwnd是动态的，是感知到的网络拥塞程度的函数

- 超时或者3个重复ack，cwnd↓
 - 超时：cwnd降为1MSS, ssthresh=cwnd/2, 进入SS阶段
 - 3个重复ack：cwnd降为cwnd/2+3, ssthresh=cwnd/2, 保持在原状态

- 否则（正常收到Ack，没有发送以上2种情况）：cwnd跃跃欲试↑

- SS阶段：加倍增加(每个ACK RTT+1)
- CA阶段：线性增加(每个ACK RTT+MSS/cwnd)

Transport Layer: 3-184

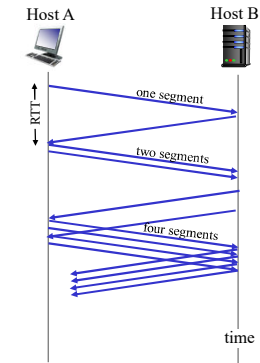
TCP 拥塞控制：策略概述

- 慢启动
 - AIMD：线性增、乘性减少
 - 超时事件后的保守策略
- 慢启动
 - 连接刚建立, $cwnd = 1 \text{ MSS}$
 - 如: $MSS = 1460 \text{ bytes}$ & $RTT = 200 \text{ msec}$
 - 初始速率 = 58.4 kbps
 - 可用带宽可能 $\gg MSS/RTT$
 - 应该尽快加速, 到达希望的速率
 - 当连接开始时, 指数性增加发送速率, 直到发生丢失事件
 - 启动初值很低
 - 但是速度很快

Transport Layer: 3-185

TCP 慢启动

- 当连接开始时, 指数型增加其拥塞窗口, 直到第一个丢失事件发生:
 - 初始化 $cwnd = 1 \text{ MSS}$
 - 每个RTT倍增 $cwnd$
 - (相当于每收到一个ACK, $cwnd$ 增加一个MSS)
- **总结**: 初始速率很低, 但是增速却是指数型的



Transport Layer: 3-186

TCP 拥塞控制: AIMD

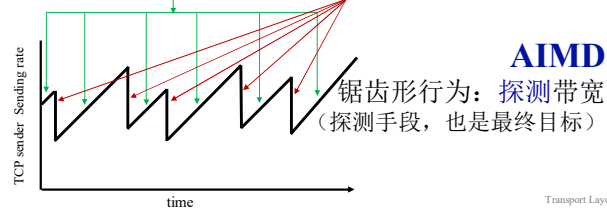
- **方法**: 发送方增加其发送速率直到网络分组丢失（拥塞）发生, 然后在发生丢失事件时降低其发送速率

Additive Increase

增加发送速率, 每个RTT增加一个MSS直到丢失被探测到

Multiplicative Decrease

在每个丢失事件发生时减半发送速率



Transport Layer: 3-187

TCP AIMD (续)

Multiplicative decrease 细节: 发送速率是

- 在通过收到三个冗余ACK探测到丢失时: 降低到一半 (TCP Reno)
- 当超时事件发生时, 丢失被探测到 (TCP Tahoe): 降低到1 MSS (maximum segment size) 每个RTT, 为什么 **AIMD**?

■ AIMD – 一个分布式, 异步算法 – 已被证明:

- 全网优化: 不拥塞同时吞吐大!
- 具备理想的稳定性

Transport Layer: 3-188

TCP: 从慢启动SS到用拥塞避免CA

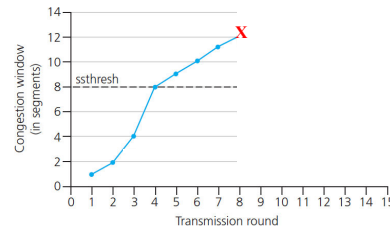
2025中科大网络

Q: 什么时候从指数型增加切换成线性增加?

A: 当 **cwnd** 变成上次超时 congestion window 的一半时

实现:

- 变量 **ssthresh**
- 一旦丢失, **ssthresh** 被设置成发生丢失事件前 **cwnd** 的一半

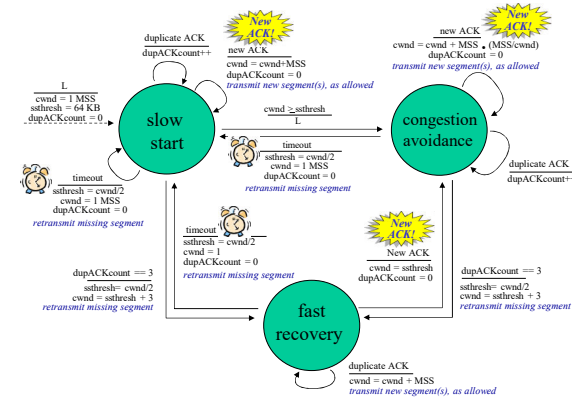


* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Transport Layer: 3-189

总结: TCP拥塞控制

2025中科大网络

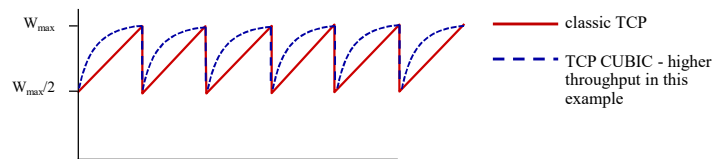


Transport Layer: 3-190

TCP CUBIC

2025中科大网络

- 有比AIMD更好的探测可用带宽的方法吗?
- 洞察/直觉:
 - W_{max} : 当拥塞丢失被检测时的发送速率
 - 瓶颈链路的拥塞状态可能 (?) 没有发生太大变化
 - 发生丢失时将发送速率砍半, 最初更快地增加到 W_{max} , 但是随后更慢地接近 W_{max}

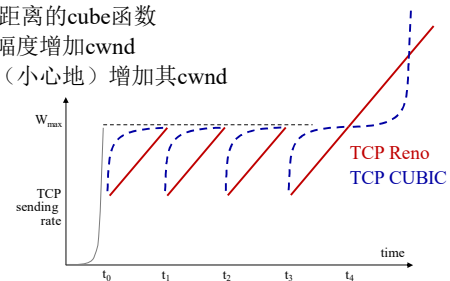


Transport Layer: 3-191

TCP CUBIC

2025中科大网络

- K:** TCP窗口大小将达到 W_{max} 的时间点
 - K 是可调参数
- 增加的 $W = \text{cube}(K - t)$
 - W 作为当前时间点和 K 的距离的 cube 函数
 - 时间点离 K 比较远时, 大幅度增加 **cwnd**
 - 当接近于 K 时, 比较缓的 (小心地) 增加其 **cwnd**
- Linux 默认配置 **cubic**, 一些流行的 Web 服务器也配置 **CUBIC**

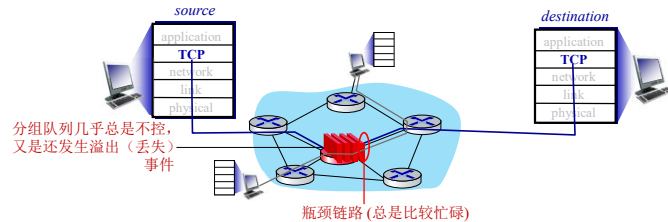


Transport Layer: 3-192

TCP 和拥塞的“瓶颈链路”

2025中科大网络

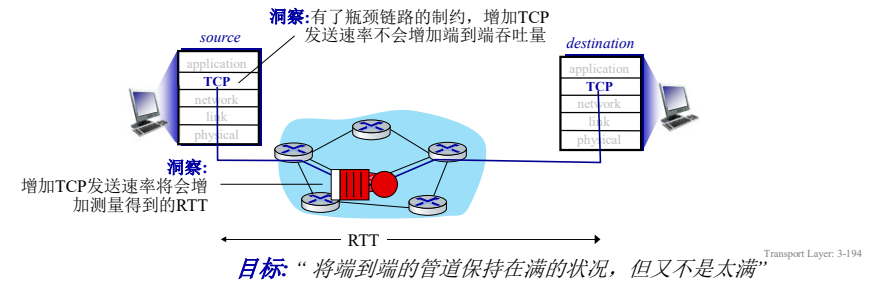
- TCP (典型, CUBIC)增加TCP发送速率, 直到在一些路由器的输出端口发生分组丢失事件: **瓶颈链路**



TCP 和拥塞的“瓶颈链路”

2025中科大网络

- TCP (典型, CUBIC)增加TCP发送速率, 直到在一些路由器的输出端口发生分组丢失事件: **瓶颈链路**
- 理解拥塞: 有利于聚焦到拥塞的瓶颈链路上



基于延迟的TCP拥塞控制

2025中科大网络

保持从发送端到接收端管道“满, 但又不是太满”: 保持瓶颈链路处在忙碌传输状态, 但是又要避免高延迟/缓存



基于延迟的拥塞控制方法:

- RTT_{min} - minimum observed RTT (uncongested path) 最小观察到的RTT, 非拥塞时
- 具有拥塞窗口cwnd的不拥塞时的吞吐: cwnd/RTT_{min}

如果测量得到的吞吐“非常接近”于不拥塞吞吐,
线性增加cwnd /* 因为路径不拥塞 */
否则: 如果测量得到的吞吐“远远低于”不拥塞时的吞吐
线性降低cwnd /* 因为路径拥塞了 */

Transport Layer: 3-195

基于延迟的TCP拥塞控制

2025中科大网络

- 拥塞控制没有诱导/强迫丢失 (用于探测可用带宽, 同时增加吞吐)
- 最大化吞吐量 (“让管道保持满的状态...”), 同时保持低延迟 (“...但不是更满”)
- 一些TCP的实现采用了基于延迟的方法
 - 部署在谷歌 (内部) 骨干网络中的BBR算法

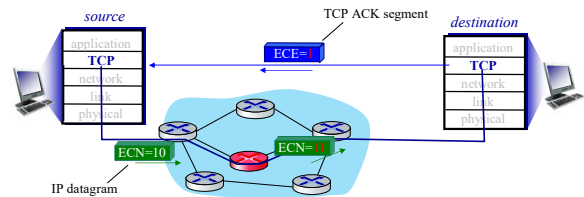
Transport Layer: 3-196

Explicit congestion notification (ECN)

2025中科大网络

TCP 部署通常实现了网络辅助的拥塞控制:

- IP头部的2个bit (ToS字段) 被网络路由器标记来指示是否发生了拥塞
 - 被网络运营商选择的标记策略
- 拥塞指示位被携带到目标主机
- 目标主机在ACK段中设置ECE bit来通告发送端拥塞情况
- 涉及到IP (IP头部 ECN标志位 位置) 和TCP (TCP头部 C,Ebit标记)

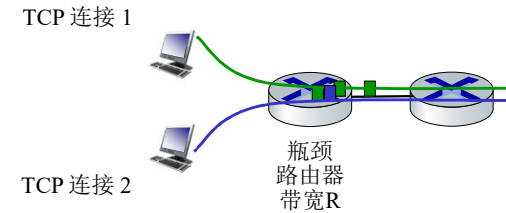


Transport Layer: 3-197

TCP 公平性

2025中科大网络

公平性目标: 如果K个TCP连接共享一个 带宽为R的瓶颈链路, 每个TCP连接获得R/K的带宽



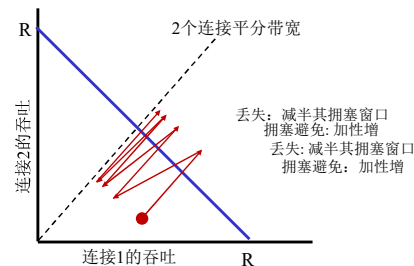
Transport Layer: 3-198

Q: TCP公平吗?

2025中科大网络

例子: 2个竞争的TCP会话:

- 当吞吐量增加时, 加性增以斜率1线上走
- 当吞吐量比例减少时, 乘性减



TCP公平吗?

A: 是的, 在理想假设下:

- 相同的RTT
- 固定数量的会话, 只有CA阶段

Transport Layer: 3-199

公平性:对所有网络应用都是“公平”的吗?

2025中科大网络

公平性和UDP

- 多媒体应用通常不运行在TCP之上
 - 不希望其速率受到拥塞控制机制的制约
- 而是在UDP之上运行:
 - 以稳定的速率发送音视频数据
 - 容忍丢包
- 没有“互联网警察”监管拥塞控制机制的使用

公平性和并行TCP连接

- 在一对主机之间应用可以打开多个并行TCP连接
 - Web浏览器采用这种方法, 例如: Rbps带宽的链路上已经有9个已经存在的连接
 - 新的应用打开1条TCP连接, 它会得到R/10的带宽
 - 如果新的应用打开11条TCP连接, 它会得到R/2的带宽

Transport Layer: 3-200

传输层：提纲

2025中科大网络

- 传输层服务
- 多路复用解复用
- 无连接的传输：UDP
- 可靠数据传输原理
- 面向连接的传输：TCP
- 拥塞控制原理
- TCP拥塞控制
- 传输层协议的演化



Transport Layer: 3-201

演化传输层功能

2025中科大网络

- TCP, UDP: 支撑互联网发展40余年的主要传输层协议，
- 针对特定场景，开发了不同“风格”的TCP

场景	挑战
长、肥管道（大量数据传输）	很多“飞行中”的分组；丢失发生会关闭管道
无线网络	由于有噪声的信道噪声的丢失，移动性，TCP将之看成是拥塞造成的丢失
长延迟链路	超长的RTT
数据中心网络	延迟敏感
背景流量	低优先级，背景TCP流量

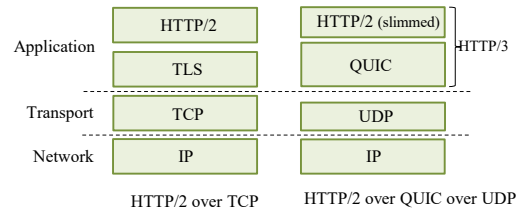
- 将传输层的功能移动到应用层次，在UDP之上实现
 - HTTP/3: QUIC

Transport Layer: 3-202

QUIC: Quick UDP Internet Connections

2025中科大网络

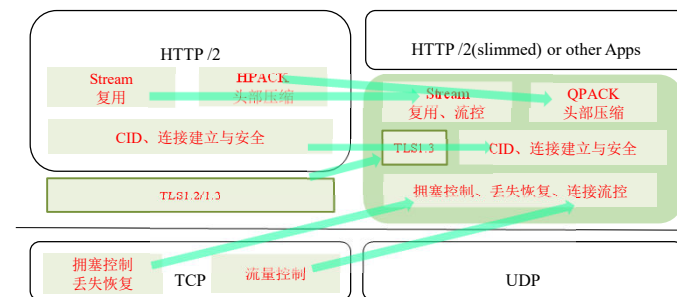
- 应用层协议，在UDP之上
 - 增加了HTTP的性能
 - 部署于很多谷歌服务器之上，很多应用支持在QUIC之上运行 (Chrome, mobile YouTube app)



Transport Layer: 3-203

QUIC: Quick UDP Internet Connections

2025中科大网络



Transport Layer: 3-204

QUIC: Quick UDP Internet Connections

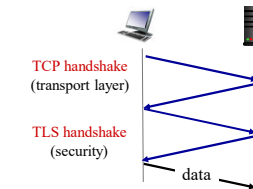
采用了我们这一章之前学习的方法：有关于连接建立，差错控制，拥塞控制等

- **差错控制和拥塞控制**：“熟悉TCP的丢失检测和拥塞控制的读者会在这里找到与知名的在TCP中的平行算法。” [from QUIC specification]
- **连接建立**：可靠性（初始序号），拥塞控制（窗口），认证（身份认证），加密状态（密钥）在一个RTT内被建立

- 多个应用级别的“流”复用于一个QUIC连接之上
 - 独立的可靠数据传输和安全性
 - 公共的拥塞控制

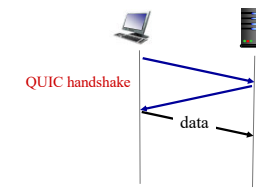
Transport Layer: 3-205

QUIC: 连接建立



TCP (可靠性即初始序号等, 拥塞控制状态即窗口大小等) + TLS (认证, 加密状态即分发密钥等)

- 2个层面的2个序列握手

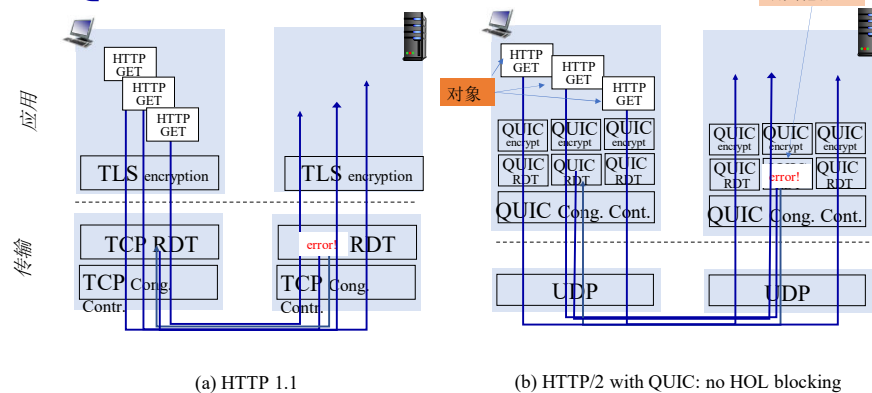


QUIC: 可靠性，拥塞控制，认证，加密状态

- 1次握手

Transport Layer: 3-206

QUIC: 流: 并行化, 没有HOL堵塞



(a) HTTP 1.1

(b) HTTP/2 with QUIC: no HOL blocking

Transport Layer: 3-207

第3章: 总结

- 传输层服务背后的原理:
 - 多路复用, 多路解复用
 - 可靠数据传输
 - 流量控制
 - 拥塞控制
- 互联网传输层协议的实例和实现
 - UDP
 - TCP

到目前为止:

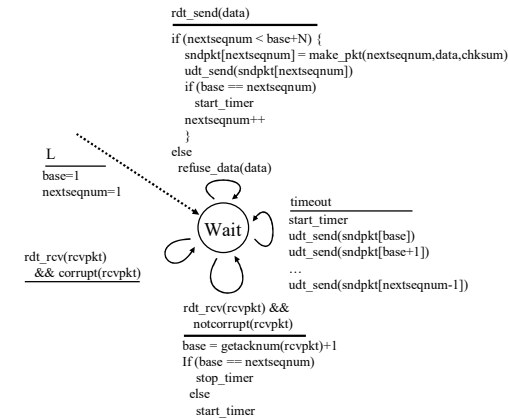
- 离开了网络的“边缘”（应用层，传输层）
- 深入到网络的“核心”
- 网络层有2章
 - 数据平面
 - 控制平面

Transport Layer: 3-208

第3章附件幻灯片

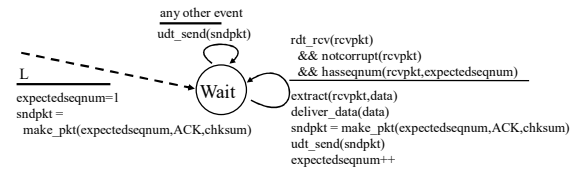
Transport Layer: 3-209

Go-Back-N: 发送方的扩展 FSM



Transport Layer: 3-210

Go-Back-N: 接收方的扩展 FSM

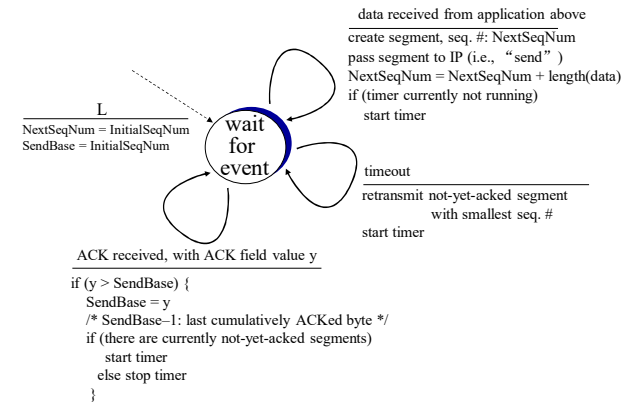


ACK-only: 总是发送对 **正确收到的+顺序到达的最高序号**的分组，给予确认

- 可能产生重复的 ACKs
- 需要记住接收窗口: **expectedseqnum**
- 乱序到达的分组:
 - 丢弃 (不缓存): **接收方没有缓存!**
 - 重新发出确认，确认具有 (最高序号+顺序到达) 的分组

Transport Layer: 3-211

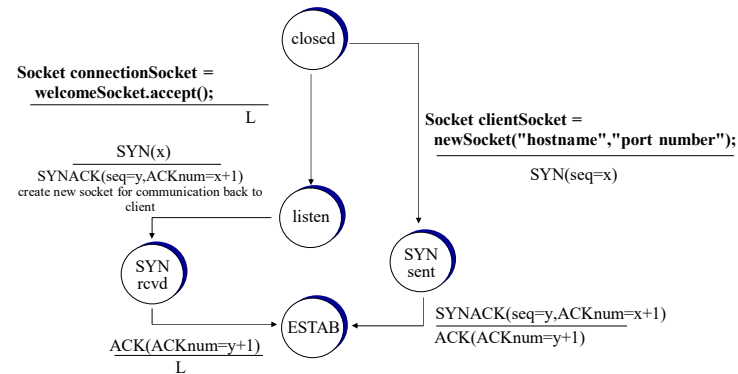
TCP 发送方 (简化版)



Transport Layer: 3-212

TCP 3次握手FSM

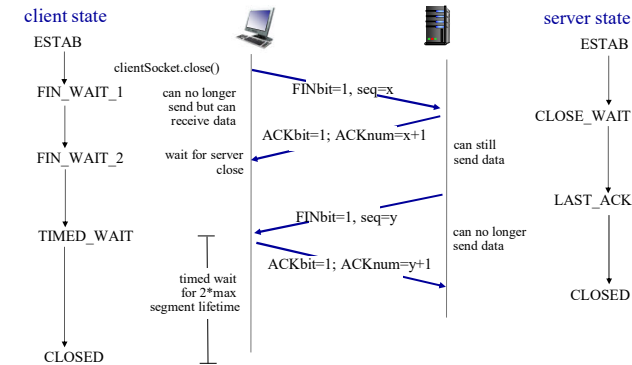
2025中科大网络



Transport Layer: 3-213

关闭TCP连接

2025中科大网络

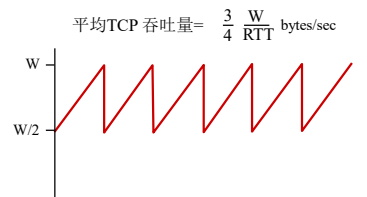


Transport Layer: 3-214

TCP 吞吐量

2025中科大网络

- 平均TCP吞吐量，是窗口大小W和往返延迟RTT的函数？
 - 忽略慢启动，假设总有数据要传输
- W: 丢失事件发生时的窗口大小 (以字节为单位)
 - 平均窗口大小 (在飞行中的字节数量) 是: $\frac{3}{4}W$
 - 平均吞吐量是: 每个RTT，发送 $\frac{3}{4}W$ (字节大小)



在“长，肥管道”上的TCP

2025中科大网络

- 例子: 1500 byte 段, 100ms RTT, 需要 10 Gbps吞吐
- 需要窗口 $W = 83,333$ 飞行中的段
- 吞吐量用段的丢失率L表达 [Mathis 1997]:

$$\text{TCP throughput} = \frac{1.22 \cdot \text{MSS}}{\text{RTT} \sqrt{L}}$$

→ 为了达到 10 Gbps吞吐量，需要段的丢失率 $L = 2 \cdot 10^{-10}$ — 一个非常小的丢失率! (以至于小于 收到干扰出错而丢失的比例，不现实)

- 需要一个能够适应RTT长、高速场景的新TCP版本

Transport Layer: 3-216